



UMEÅ UNIVERSITY

An abstract, high-contrast image with swirling patterns in shades of blue, green, and orange, resembling a microscopic view of a biological structure or a digital data visualization. The text is overlaid on this image.

OPTIMIZING DISTRIBUTED TRACING OVERHEAD IN A CLOUD ENVIRONMENT WITH OPENTELEMETRY

Elias Norgren

Master Thesis, 30 credits

MASTER OF SCIENCE PROGRAMME IN COMPUTING SCIENCE

2024

Abstract

To gain observability in distributed systems, some telemetry generation and gathering have to be implemented. This is especially important when systems have layers of dependencies on other microservices. One method for observability is called distributed tracing. Distributed tracing is the act of building causal event chains between microservices, which are called traces. Finding bottlenecks and dependencies within each call chain is possible with the traces. One framework for implementing distributed tracing is OpenTelemetry. The developer must determine design choices when deploying OpenTelemetry in a Kubernetes cluster. For example, OpenTelemetry provides a collector that collects spans, which are parts of a trace from microservices. These collectors can be deployed one on each node, called a daemonset. Or it can be deployed with one for each service, called sidecars. This study compared the performance impact of the sidecar and daemonset setup to that of having no OpenTelemetry implemented. The resources analyzed were CPU usage, network usage, and RAM usage. Tests were done in a permutation of 4 different scenarios. Experiments were run on 4 and 2 nodes, as well as a balanced and unbalanced service placement setup. The experiments were run in a cloud environment using Kubernetes. The tested system was an emulation of one of Nasdaq's systems based on real data from the company. The study concluded that having OpenTelemetry added overhead / increased resource usage in all cases. Having the daemonset setup, compared to no OpenTelemetry, increased CPU usage by 46.5 %, network usage by 18.25 %, and memory usage by 47.5 % on average. Sidecar did, in most cases, perform worse than the daemonset setup in most cases and resources, especially in RAM and CPU usage.

Acknowledgements

I am grateful to my university supervisor, Per-Olov Östberg, for his advice and insights that significantly elevated the academic quality of this work. I also wish to sincerely thank my manager and supervisor at NASDAQ. Special thanks to my colleague, Mert, whose collaboration has been essential in achieving even better results. Finally, I am thankful to my girlfriend, Amrei, for her emotional support throughout this process.

Contents

1	Introduction	1
1.1	Background	2
1.2	Purpose	2
1.3	Research Question	3
1.4	Methods Used	3
1.5	Limitations	4
2	Related Work	5
3	Theoretical Background	7
3.1	Distributed Systems	7
3.2	REST	7
3.3	Microservices	7
3.4	Containers	8
3.5	Kubernetes	9
3.6	The Three Pillars of Observability	10
3.7	Distributed Tracing	10
3.8	OpenTelemetry	12
3.9	Collector Deployment	14
4	Method	16
4.1	Kubernetes	16
4.2	Micro Service Setup	17
4.3	OpenTelemetry	20
4.4	Collector Placement	21
4.5	Load Generator	25
4.6	Test Cases	25
4.7	Metric Collection	26
5	Results	27
5.1	Time Taken	27
5.2	2 Nodes	29
5.3	4 Nodes	33
6	Discussion	38
6.1	OpenTelemetry Overhead	38
6.2	Sidecar and Daemonset	38
6.3	2 and 4 Nodes	40
6.4	Balanced and Unbalanced	40
7	Conclusion	42
7.1	Future Work	42
7.2	Recommendations	43

List of Figures

1	Services generating and exporting spans	2
2	Container and VM difference	9
3	Kubernetes example architecture	9
4	Services example	11
5	Tracing example	12
6	Collector architecture	14
7	Daemonset vs Sidecar	15
8	Collector load balancer deployment	15
9	Calls per service and service connections	17
10	Node placement 4 nodes	18
11	Node placement 2 nodes	19
12	Received requests per balance	19
13	Invoked requests per balance	20
14	System architecture	21
15	Traces for sidecar 4 nodes	22
16	Traces for sidecar 2 nodes	23
17	Worker 4, balanced and expanded	23
18	Traces for daemonset 4 nodes	24
19	Traces for daemonset 2 nodes	25
20	2 nodes time taken	28
21	4 nodes time taken	28
22	2 nodes balanced CPU usage	29
23	2 nodes unbalanced CPU usage	30
24	2 nodes balanced network usage	31
25	2 nodes unbalanced network usage	31
26	2 nodes balanced memory usage	32
27	2 nodes unbalanced memory usage	33
28	4 nodes balanced CPU usage	34
29	4 nodes unbalanced CPU usage	34
30	4 nodes balanced network usage	35
31	4 nodes unbalanced network usage	36
32	4 nodes balanced memory usage	37
33	2 nodes unbalanced memory usage	37

List of Tables

1	W3C TraceContext header	13
2	CloudGuru resource units for servers	16

1 Introduction

This study analyzes the performance overhead of distributed tracing in distributed systems. Distributed tracing measures and observes latency at different locations as requests are propagated within a distributed system. The tool used for implementing distributed tracing is OpenTelemetry. OpenTelemetry is an open-source framework for collecting traces, metrics, and logs in distributed systems. This is implemented by manually instrumenting the code to trace or by automatic instrumentation. Automatic instrumentation can be done by either injecting code or dependencies into the application. It requires that the application is written in one of the supported programming languages. This report will refer to a service as the component or application running a microservice. The term service should not be confused with the Kubernetes service. Each call to a service is called a span; one or more spans make up a trace, which is the whole chain of calls in a tree-like form. Each service injects a trace context in the header of each request, which is used for later reference. When each service is finished, they report their span to a collector. The system contains zero or more collectors who receive, process, and export traces to a back-end, this can be seen in Figure 1. The collector can be deployed in Kubernetes using different methods. It can be deployed as a daemonset, meaning each node has a collector to which the services on that node report spans. It can be deployed as a sidecar, meaning each pod has its own collector. It can also be deployed with a load balancer that can route spans to an arbitrary collector in the cluster. How to deploy the collectors is up to the one implementing the system. This study aims to investigate the differences in resource overhead when using different collector deployment strategies combined with different node setups and service placements.

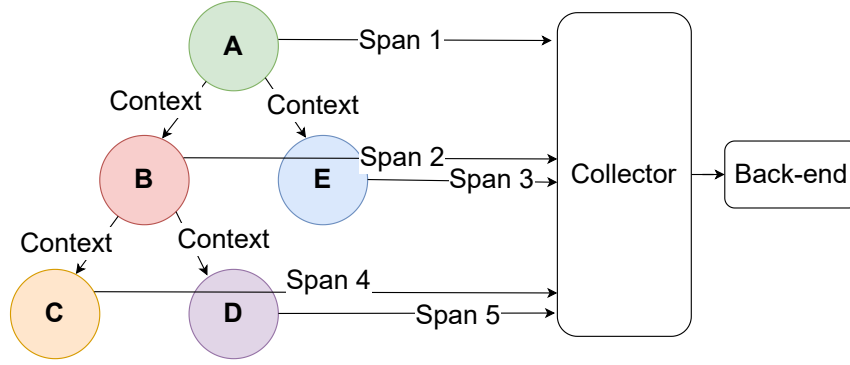


Figure 1: The figure illustrates how five different services that have some relationship generate and export spans to a collector and finally to a back-end. In each invocation to another service a context is propagated to keep track of which trace the span belongs to.

1.1 Background

Microservices are self-contained, stateless applications that communicate via messages and expose some requestable operations. Distributed systems that use microservice architectures have become increasingly popular compared to their monolithic counterparts. Microservice architectures and other cloud-native applications are popular choices when developing software systems. However, this does come at the cost of less observability as services may be spread across several machines. When multiple microservices interact, it gets harder to track the flow of execution and determine the bottlenecks. Therefore, different observability frameworks have been developed to help coordinate the gathering and generation of telemetry in distributed systems.

1.2 Purpose

This project will be conducted in cooperation with Nasdaq. The study aims to answer Nasdaq's questions on how distributed tracing can be implemented with minimal overhead and how much overhead it entails. NASDAQ is developing clearing house software solutions for their customers. A clearing house acts as an intermediary in financial markets, facilitating transactions between buyers and sellers of various securities. It is important to validate these transactions and ensure their accuracy and completion. When trading with futures, clearing houses take on the opposite positions of trades, effectively buffering buyers and sellers against counterparty risk. By doing so, they help mitigate market risk, contributing to the financial system's stability. In practice, this means matching buyers and sellers while also calculating trade

risk. These operations are done billions of times per day (Nasdaq 2021). Therefore, NASDAQ is interested in tools such as OpenTelemetry to help improve observability in their systems. One such method is distributed tracing, which is necessary when migrating a system to a cloud environment.

This project aims to emulate one of Nasdaq’s products for testing and experimentation. The product emulated consists of several connected microservices. Some perform more frequent and heavy computations, like risk calculations and trade management operations. The system implemented in this report tries to resemble the number of services, and the load to each service is exposed. The emulation was based on real data given from Nasdaq, such as the system design, workloads, and the data sent between services. Business logic has intentionally been left out due to the project’s limitations. The names of services have been changed due to confidentiality issues.

1.3 Research Question

Is there a measurable difference in resource overhead, such as CPU utilization, RAM consumption, and network usage for different node and service setups when using different OpenTelemetry deployments for distributed tracing, and how do the trade-offs among these impact deployment strategies?

1.4 Methods Used

A literature study was performed to understand the subject better. Several experiments were run to answer the research question. A Kubernetes cluster was set up with configurable REST servers to run the experiment. The servers were automatically instrumented to perform tracing, but tracing could be turned off if the user wanted. These servers were linked together to make up a call chain between services. The system was built so that an arbitrary setup of services would be easy to configure. The system would also contain one or more collector services that collect tracing from services and export them to a back-end. The system would be easily configurable when it comes to collector deployment methods. Logs from each service can be used to measure latency and throughput. Prometheus was used to measure CPU, memory, and network usage. The experiment setup varied with 2 and 4 nodes. Two different service placement setups were also examined: one balanced, where the amount of requests sent between nodes was minimized, and one unbalanced, where requests between nodes were maximized.

1.5 Limitations

The main limitation of this study was the resource limits from the cloud provider Cloud Guru. The highest amount of nodes was 4, and each node had limited CPU and RAM capacity. Having more resources would allow for more realistic testing with a more powerful cluster. Another limitation is the lack of control of VM placement in the cloud cluster and potential interference from other VMs in the cloud, which is out of control from the cloud user.

2 Related Work

Aguilera et al. (2003) described the problem that arises with multi-tiered web-based applications. They explained that these applications do get harder to debug and analyze as they grow. They proposed a solution to this problem called the black-box approach. An approach that tried to gather the causal traces in the system to identify bottlenecks. Their approach was to implement a passive tracing algorithm, which requires minimal knowledge of the application and system. It should also require no modification to the application while not impacting system performance significantly. They successfully implemented two different algorithms that achieved this. One for RPC-based systems and one for free-form message-based applications. They concluded that both algorithms could achieve passive tracing, but it did add performance overhead to the system's CPU and memory usage. In contrast to Aguilera et al., this study focuses more on the performance overhead with different trace-collecting methods. The referenced paper does not analyze this logistical problem but instead focuses on how to implement trace generation algorithms.

A study was performed by Reichelt, Kühne, and Hasselbring (2021), where a performance comparison was made between the tracing frameworks InspectIT, Kieker, and OpenTelemetry. The study configured InspectIT and OpenTelemetry to export directly to a back-end framework called Zipkin. They measured the latency of calls when the call tree depth increased from 0 to 130 and concluded that Kieker had the lowest latency, followed by OpenTelemetry and, lastly, InspectIT. The referenced study did not use any collector, which is discouraged by OpenTelemetry. The collector is recommended as it allows fast offloading of data and mechanisms like retries, batching, and encryption (OpenTelemetry 2024c). They also did not test with different amounts of nodes and different service placements. In contrast to the referenced study, this study aims to cover the performance impacts of collector usage and placement.

A study performed by Cinque, Corte, and Pecchia (2022) developed a monitoring framework called MetroFunnel. MetroFunnels tracing worked like OpenTelemetry using a method the authors called passive tracing. Passive tracing is when tracing is performed non-intrusively by injecting data into the REST headers. By combining logs and tracing, the authors created a tool to help developers understand and analyze telemetry from services in a Kubernetes cluster while simultaneously reducing log sizes. They concluded that it was possible to reduce log size by 59 % at the cost

of 3.3 % performance overhead. In contrast to the referenced study, this study uses OpenTlemetry instead of MetroFunnel, it also focuses more on collector performance on different Kubernetes setups.

Another study by Perdanaputra and Kistijantoro (2020) extended the MetroFunnel project to work for gRPC, called Inkle. They motivated the use of gRPC because it performs better than HTTP/1.1. The system's overall architecture included an Inkle agent deployed as a daemonset on each node, which would intercept gRPC packets, perform tracing injections, and write logs to a file. Another daemonset, Filebeat, would collect data from the log file and export it to a back-end. They concluded that the implementation increased the average CPU usage by 8.8 % with elastic stack and 3.5 % without elastic stack. The memory usage increased by 959 MB with elastic stack and 378 MB without. In contrast to the referenced study, this study uses OpenTelemetry, implemented on REST based microservices, instead of gRPC. OpenTelemetry also provides the necessary mechanisms and standards for collecting and processing the traces in the distributed system. As opposed to MetroFunnel which also depended on elastic stack, Filebeat and Inkle.

Rabkin and Katz (2010) described log collection problems that appear in large-scale systems. They developed a system called Chukwa, a sub-system of Hadoop. Chukwa would place an agent on each node to gather logs. The agents would then send their logs to a collector which wrote them to the file system. They concluded that their approach could scale effectively with only a small overhead being added. They were able to exceed the highest cluster-wide log gathering reported in the literature at that time. The referenced study differs from this one as it was built for log gathering and not tracing. However some of the same mechanisms were used, such as different collector deployments and how many agents on each node.

3 Theoretical Background

This chapter describes and explains the background for the key concepts used in this study, including both the technical and reasoning behind them.

3.1 Distributed Systems

Historically, during the modern computer era, ca. 1985, computers were heavy and not as powerful as they are today. Computations were often carried out on a single core with non-parallel execution without being connected to a network. The rise of high-speed networks and multicore processors has made switching to parallel execution more attractive. Now, computations can be run in parallel on a single computer and in coordination with other computers in a network (Van Steen and Tanenbaum 2016). There exist different definitions of what a distributed system is, but according to Van Steen and Tanenbaum, it reads as follows “A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system.”

3.2 REST

Representational State Transfer (REST) is a set of principles guiding API development, particularly for complex networks like the internet. It enables easy implementation and modification, ensuring visibility and cross-platform portability. RESTful APIs adhere to REST principles, providing uniform interfaces, statelessness, layered systems, cacheability, and optional code-on-demand functionality. The benefits of RESTful APIs include scalability, flexibility, and independence from underlying technologies. RESTful API communication involves clients sending requests containing resource identifiers, methods (e.g., GET, POST, PUT, DELETE), HTTP headers, data, and parameters, which are processed by servers and return responses indicating success or failure (Amazon web services n.d.).

3.3 Microservices

A microservice is a self-contained, stateless unit that communicates through messages. It should expose requestable operations regarding some tasks, but not more than that. For example, a calculation service should expose an endpoint to do some

calculations. It should not provide other loosely coupled functionality like graphing or displaying the result (Dragoni et al. 2017, p. 2). The microservice architecture is a popular trend in application development. The architecture is a distributed system where all the components are microservices. The architecture originated from service-oriented architecture (SOA). By partitioning the system into smaller pieces, microservice architecture emphasizes highly maintainable and scalable code. Troubleshooting and scaling the project also becomes more effortless. Microservices can be implemented as web-based REST servers. This means that a microservice architecture can be made up of Microservices written in several languages. If the programming language supports the creation of a REST server, it can expose a microservice (ibid.).

3.4 Containers

The word container is taken from the shipping industry, where containers ship goods. In the software context, containers are used to ship and run software. A container is a lightweight standalone package that contains the runtime, software, and libraries needed to run processes in isolation. A container is launched from an image, the container's building block. The container can then run one or more processes. Like a shipping container, the container runtime does not care or know what is inside the container. A common container runtime is the docker engine. Some examples of operations that the runtime can perform on the container are to create or destroy it, start it, or stop it. The image contains the image format, a set of standard operations, and an execution environment (Turnbull 2014, p. 13).

Unlike a virtual machine (VM), a container does not virtualize hardware but instead virtualizes the operating system; the architecture can be seen in Figure 2. This difference makes containers more efficient regarding CPU, RAM, and disk I/O. This improved performance does come at the cost of lower isolation (Potdar et al. 2020). The improved performance makes scaling up and down a container easier than a VM. Combining performance, scalability, and portability features makes containerization an attractive option over VMs as most public cloud providers need virtualization to power their infrastructure. However, there are cases where a VM is suitable over containers, for example, when dealing with crucial business data, as the VM gives higher isolation (Joy 2015). It is also possible to use a combination of both. For example, a server might run a number of VMs, which in turn holds some containers.

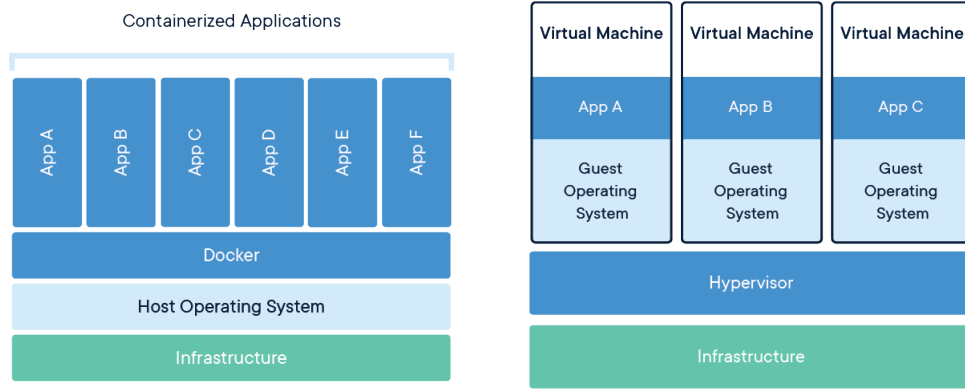


Figure 2: The virtualization difference between containers and VM's. The containers run directly from the Docker engine, whereas a VM virtualizes hardware from the hypervisor (Wikimedia Commons 2022).

3.5 Kubernetes

Since containers are fast and flexible, they are also fragile and short-lived. The container does not have a firewall or other protective systems engaged that the VM has. With this in mind, containers benefit from running in orchestrated and scheduled clusters (Rensin 2015, p. 3). When a container starts malfunctioning at larger scales, it is cleaned up and restarted. Kubernetes is one such container orchestrator, open-sourced by Google from its internal orchestration system Borg (ibid., p. 5). Kubernetes does not schedule and run containers directly but instead run pods. A pod is a collection of containers with the same IP and file system (ibid.). An example of how a Kubernetes cluster setup can look can be seen in Figure 3.

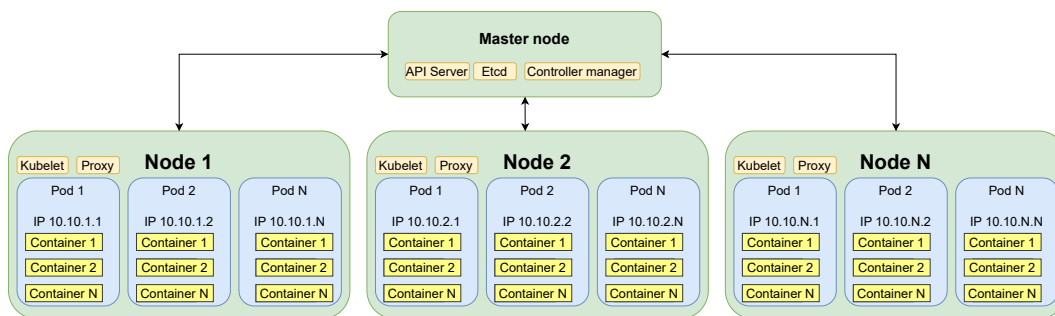


Figure 3: A Kubernetes example setup where there is one master node connected to several worker nodes. Each node holds one or more pods, each containing one or more containers. The IP address for each pod is based on which node and pod it is.

A typical Kubernetes cluster consists of a master node and one or more worker nodes. A node, in this context, refers to a physical machine. The following components define the master node in Kubernetes:

1. API server, responsible for managing nearly everything in the cluster.
2. Etcd, a key-value store that keeps the current state of the cluster.
3. Scheduler and controller manager, who schedules pods containing containers.

A worker node consists of the following components:

1. Kubelet which is a daemon that answers requests from the master to manage containers, for example, create or delete containers.
2. Proxy to separate the IP from the containers and the service.
3. Container Advisor (cAdvisor), that collects and exports information about containers.

Kubernetes exists in different distributions; one of them is K3s, which this thesis focuses more on. The K3s distribution created by Rancher Labs is a lightweight, slimmed-down variant compared to K8s (Kubernetes). This comes at the cost of not having a distributed database implemented (Slingerland n.d.).

3.6 The Three Pillars of Observability

Metrics, logs, and tracing are coined as the three pillars of observability. Together, they make it possible to observe a distributed environment from multiple angles. Metrics are numerical values gathered at regular intervals from one or more systems. These numbers can then be aggregated to form a comprehensive view of the system's status. This gets more necessary as the system grows. Out of the three pillars, metrics are the easiest to collect and store. The second pillar is logging, a detailed record that captures every action and event within a system, typically in a structured text format. Logs can be printed directly when an event happens or stored for future analysis. The third pillar is traces, which can help understand latencies in transactions between services and can be used to identify bottlenecks or find root cause errors. A trace comprises one or more transactions, forming a causal and logical flow of the request through the system (Chakraborty et al. 2021, p. 26).

3.7 Distributed Tracing

Distributed tracing is when tracing is applied in a distributed environment. A distributed trace is a set of related, causal spans. A span is a single event within a

service. If one service calls another, a parent-child relationship is made between the spans, creating a single trace. A trace consisting of spans can be seen in Figure 5. A span is created when an event occurs in a service instrumented with tracing, usually when it is invoked or invokes another service. The span typically contains the ID of the whole trace, the ID of the current span, and the ID of the parent span unless the current span is the root span. It also contains the start and end times of the span. If the service being called calls another service, some information or context has to be propagated to the child to keep the causal information between events. This can be seen in Figure 4. For the second service to be able to correlate itself to the right trace, the ID of the trace has to be propagated. The service being called also needs to know which span is its parent, therefore a parent span ID is also propagated to the child. When the trace is completed, the services contain the individual spans that, when collected, can form the whole trace. The trace can then be used to determine bottlenecks and latencies in the system. However, adding propagation meta-data between service calls and collecting spans does add extra overhead to the system (Bento et al. 2021, p. 3). An example of distributed tracing can be seen in Figure 4, where five services, A, B, C, D, and E, with some nested calls included. Using distributed tracing a trace can be created after collecting all the spans from the services, the trace can be seen in Figure 5.

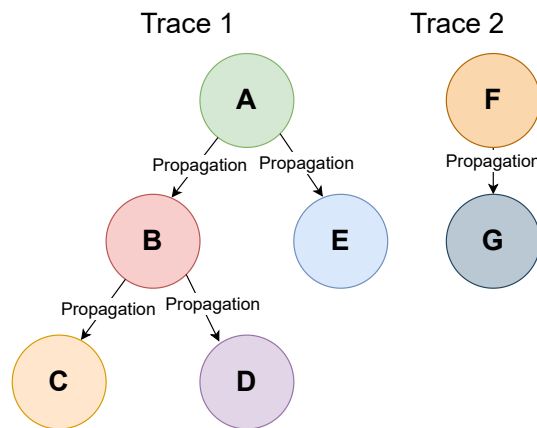


Figure 4: An example of services calling each other, creating two different spans and propagating the span context in each call. In Trace 1, service A calls B, B calls C and D. At last, A calls E.

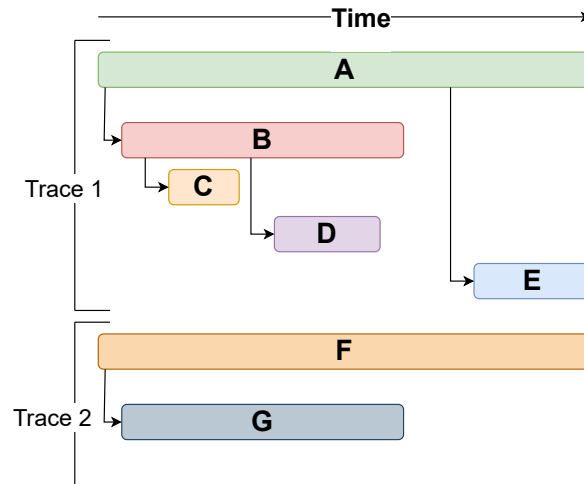


Figure 5: An example of how the spans together make up two traces and how it can be visualized. The time moves from left to right, and each block is an individual span. The arrows indicate how each service was called.

In large-scale systems, all the spans collected can create large volumes of data that need storing and overhead, which might affect the system’s performance negatively. Therefore, sampling is usually used when collecting traces. Sampling is when only a subset of all the traces are collected. The first sampling method is head-based sampling, where a decision is made at the beginning of the trace that, with some probability, this trace gets sampled. The other method is tail-based sampling. Then, the decision is made at the tail of the trace, based on some criteria like latencies of spans, errors, or other specific attributes (OpenTelemetry 2023b).

3.8 OpenTelemetry

OpenTelemetry is one of many frameworks that enable developers to collect telemetry data such as metrics, logs, and traces in distributed systems. The system is open-source and vendor-agnostic, meaning it can be connected to any other back-end if it supports one of the protocols. As OpenTelemetry only generates and collects telemetry from the services, it must be connected to some backend for observability and storage. OpenTelemetry was designed this way to prevent vendor lock-in while only learning one API (OpenTelemetry 2024i).

There are two main ways to implement OpenTelemetry in a service: automatic and manual instrumentation. If the application is written in .NET, Java, JavaScript, PHP, or Python, then OpenTelemetry provides automatic instrumentation. The automatic instrumentation mechanism may vary between languages. Usually, this is done

by code and dependency injection into the application. With this method, no code changes to the existing codebase are necessary for gathering telemetry (OpenTelemetry 2024b). The second method for instrumentation is manual instrumentation. This involves changing the application code to perform telemetry generation and export to a collector. This includes importing the OpenTelemetry API to the code, writing the code to perform telemetry generation, and exporting (OpenTelemetry 2023a). The OpenTelemetry operator, which is part of the OpenTelemetry toolkit, is used to implement automatic instrumentation in Kubernetes. The operator manages the collectors deployed in the cluster and injects an init container to pods annotated for instrumentation. This init container injects the necessary code for tracing (OpenTelemetry 2024g).

When implementing tracing with OpenTelemetry, some context has to be propagated between services to correlate spans to a trace. The standard propagation format that OpenTelemetry uses is W3C TraceContext. The W3C TraceContext contains the following information:

Table 1 The trace context that gets injected into the HTTP header when services call each other. The version tells which version of the protocol is being used. The trace ID is there so that spans can be put into the correct spans in the collector. The parent ID is used to create the causal relation when aggregating spans. Trace flags are used to determine different options, such as sampling.

Version	1 Byte
Trace ID	16 Bytes
Parent ID	8 Bytes
Trace flags	1 Byte

The version flag contains the version of the format. The trace ID is the ID of the whole trace. The parent ID is the parent span of this span. Trace flags are an 8-bit boolean mask representing, for example, sampling options. This is then encoded into a string with the following format: “traceparent: version-traceID-parentID-traceFlags.” This injects an extra 40 bytes into the HTTP header, propagating between services for each call (W3C 2021).

When the service is done with its call, and the telemetry generation is finished, it needs to be exported to either a collector or a backend immediately. The collector is built like a pipeline, where the telemetry passes from a receiver to a processor and then to an exporter. The receiver and exporter can be configured with popular proto-

cols such as OpenTelemetry Protocol (OTLP), Jaeger, and Prometheus. This makes it easy to use different back-ends for different types of telemetry. For example, metrics might be sent to a Prometheus server using the Prometheus protocol, whilst spans are sent to a Jaeger back-end using the Jaeger protocol. After the telemetry has been received, it is passed to a processor, which can modify or transform the data. An example would be sampling for tracing. The processor can easily be changed to any of the available processors. The telemetry is then sent to an exporter, which exports the telemetry to any back-end of choice. Like the receiver, the exporter can be configured with the same protocols (OpenTelemetry 2024d). The structure of the collector, where services export to a collector, can be seen in Figure 6. The flexible configuration of the collector allows developers to chain collectors or configure them according to their preferences. This enables the creation of service-specific collectors.

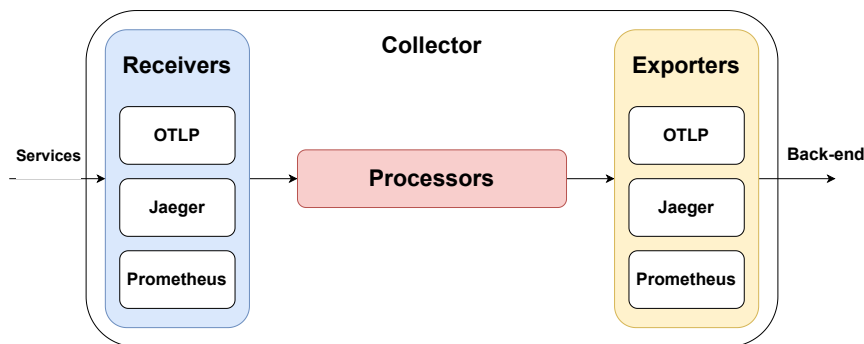


Figure 6: *The architecture of the collector. The services export their telemetry to the collector’s receiver, which sends them through a processing pipeline and then to the exporter, which exports telemetry to a back-end. The receivers and exporters support a variety of communication protocols.*

3.9 Collector Deployment

The collector can be deployed in three main ways. The first way is no collector at all. The most straightforward setup would be not to use any collector but to export trace data directly from the services to the back end. However, this adds strong coupling and requires code changes if processing, ingestion, or collecting changes (OpenTelemetry 2024f). The second way of deploying is as a sidecar or daemon set. A sidecar is when a collector is deployed with each pod that exports telemetry. A daemon set would be to deploy a collector to each node in the cluster (OpenTelemetry 2024a). The difference between the methods is that a sidecar would require more resources as there would be more instances running in parallel. However, each service would get its dedicated collector, which means less congestion. This also makes it

easier to create custom collector configurations for different services. Figure 7 shows an example of a setup with a daemon set and a sidecar. The third deployment option is with a load balancer. This means that all exporting services contact a single load-balanced collector that distributes the calls among the collectors. The routing is performed by hashing each span's trace ID to determine which collector it should be sent to. The disadvantage of this configuration is that there might be more network traffic as pods might export to collectors on other nodes. This does, however, introduce one more part to the system that might fail (OpenTelemetry 2024e). Figure 8 shows an example of a setup like this. It is also possible to create combinations of these configurations. For example, one could start by sending spans to a sidecar or daemonset collector, followed by a load balancer.

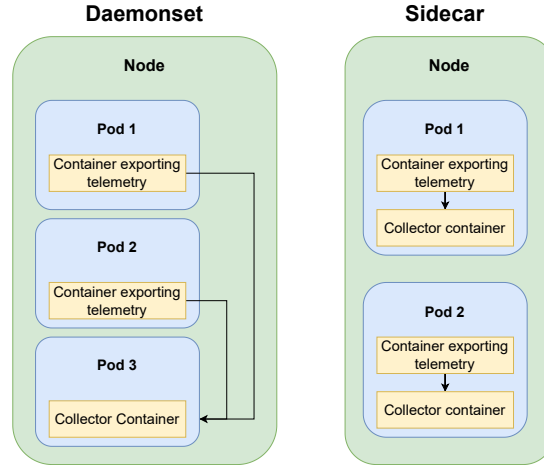


Figure 7: The difference between daemon set and sidecar deployment methods. In the daemon set, a collector is deployed at each node. With the sidecar, a collector is run within all nodes using exporting telemetry.

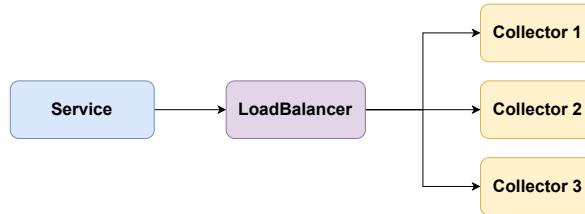


Figure 8: The collector deployed with a load balancer that distributes the exporting requests to some arbitrary collector in the cluster.

4 Method

This chapter explains the methodology of the experiments. It explains how the experiments and services were placed and set up. How much load each service and node was given is presented for each setup and resource. The system emulated was built based on the data given by Nasdaq. The two deployment methods, sidecar, and daemonset, were tested in four different environments. The tests were run on two and four nodes and on an unbalanced and balanced service setup. Where the balancing was based on the number of calls each service made between nodes. The different amount of nodes is tested because the behavior of the OpenTelemetry tracing might change when the cluster scales up or down. The balanced setups are tested since this might create imbalances in resource usage, which might affect the tracing.

4.1 Kubernetes

To emulate the real-world scenario as closely as possible, Kubernetes was chosen as a container orchestration framework. Kubernetes also comes in handy when different tools, such as Prometheus for metrics scraping across nodes, need to be used. It was also used to help place pods on specific nodes easily, consistently, and configurable. Kubernetes was chosen as it is the most well-known and used container orchestration framework to this date (Walker 2024). The Kubernetes distribution was K3s as it is lightweight and fast to deploy, as discussed in Section 3.5. CloudGuru provided the servers used to run the experiments. When creating a server in CloudGuru, a number of units must be provided for each server. Table 2 shows how much CPU and RAM each unit in CloudGuru gives.

Table 2 CloudGuru resource units for servers

Units	CPU	Memory
1 Micro	AMD EPYC 7571 (2 cores) @ 2.199GHz	1 GiB
2 Small	AMD EPYC 7571 (2 cores) @ 2.199GHz	2 GiB
3 Medium	AMD EPYC 7571 (2 cores) @ 2.199GHz	4 GiB
4 Large	AMD EPYC 7571 (2 cores) @ 2.199GHz	8 GiB

Two different Kubernetes setups, consisting of two and four nodes, were created. The two-node setup consisted of two large units, and the four-node setup consisted of one medium and three small units. Due to limited resources, the control plane had to be placed with a worker node. The Kubernetes nodes were placed in a NAT network to reduce network interference.

4.2 Micro Service Setup

Based on real data from Nasdaq, thirteen microservices were created and chained together. The services were deployed as pods in Kubernetes. Each microservice called one or more other services or only returned an answer. The connections of each service were made to resemble the Nasdaq system as closely as possible. Since recursion would occur if cycles were made, the connections were created as a directed acyclic graph with one root service. Each call between each service was deterministic, which made each call to the root node a consistent amount of work for each call. The connections between each service and how many times each calls another service per API-gateway request can be seen in Figure 9.

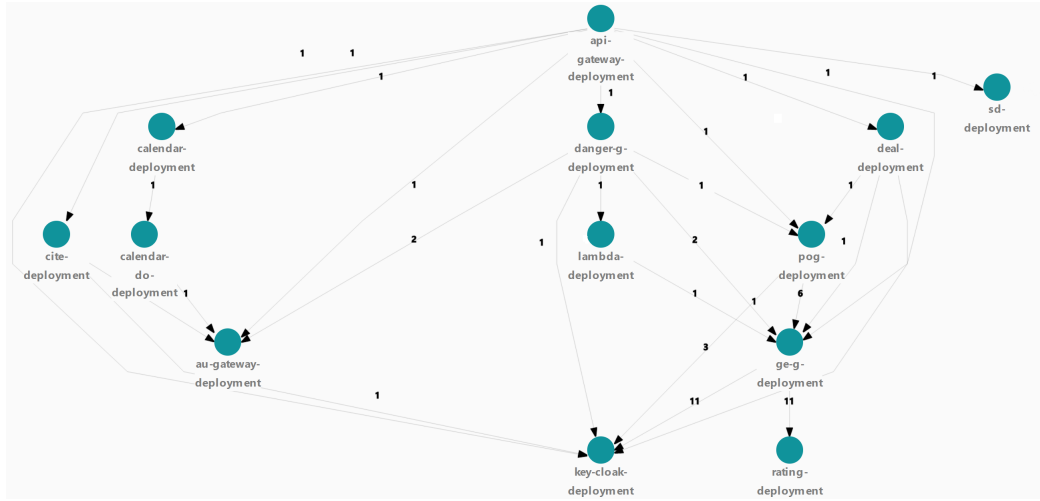


Figure 9: The API gateway is the root node to which all initial calls are made. It then calls other services, which in turn calls other services until a leaf is reached. The number on each arrow is how many times that service calls another for each call to the API gateway. Each node in the graphs is a pod in Kubernetes

4.2.1 Node Placement of Services

For each node setup, two nodes and four nodes, two additional setups were created, which were the placement of services on the nodes. These two setups are called balanced and unbalanced. The balancing that was kept in mind when placing the services for the balanced setup was to reduce the total amount of service calls between nodes, creating a network-balanced setup. The unbalanced setup tried to maximize the amount of service calls between nodes. By placing a service on a different node, calls from or to that node might increase or decrease. This is done to strengthen the hypothesis by testing in multiple environments. This would, however, lead to a CPU

imbalance for the balanced setups. The placement of services for four nodes can be seen in Figure 10, and the placement for two nodes can be seen in Figure 11. The graphs also show the distribution of how many requests each node will have to handle for each request to the cluster. These two setups make a big difference in how many requests are being received and invoked to a service on another cluster. The distribution of how the requests are made between nodes can be seen in figures 12 and 13. Note that on two nodes, the incoming call from the load generator is also accounted for in the requests received. This explains why, on two nodes, the received requests do not exactly equal invoked requests.

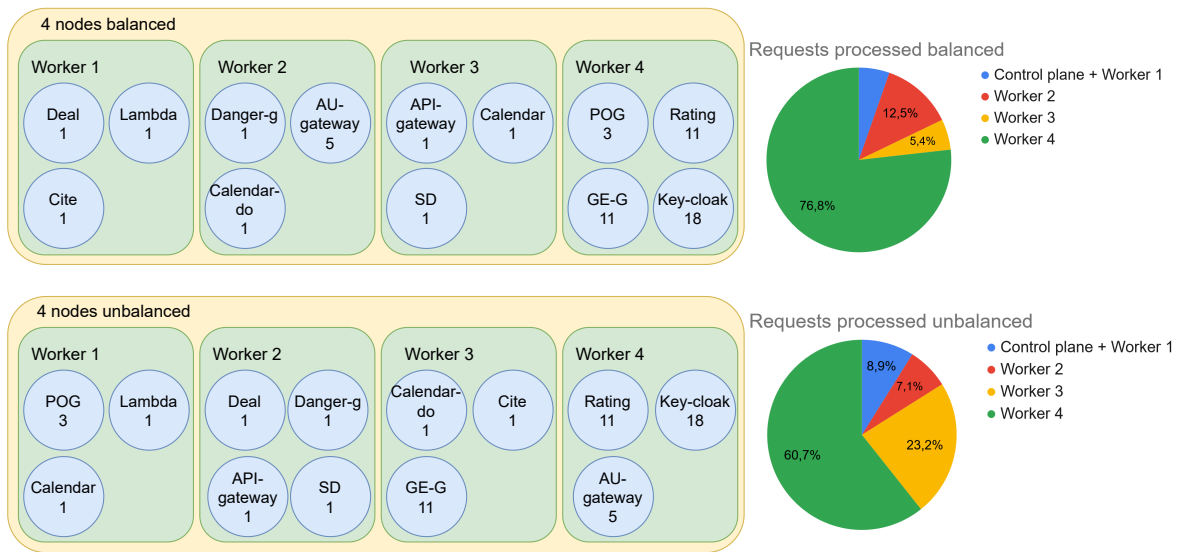


Figure 10: Service placement on workers on 4 nodes, balanced and unbalanced. Each node in green contains the given services, which are the blue circles. The number in the circle represents how many requests that service has to process per request to the API gateway. The pie chart describes how many percent of each node will have to work per request.

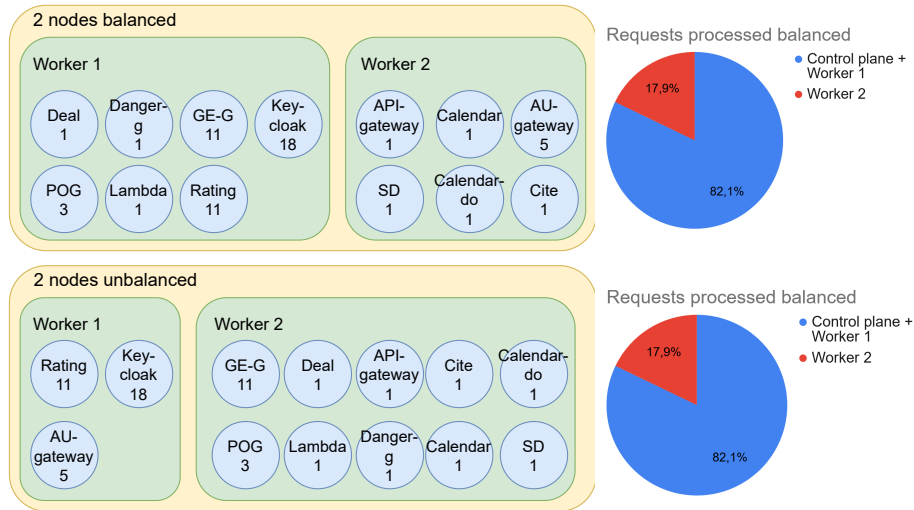


Figure 11: Service placement on workers on 2 nodes, balanced and unbalanced. Each node in green contains the given services, which are the blue circles. The number in the circle represents how many requests that service has to process per request to the API gateway. The pie chart describes how many percent of each node will have to work per request

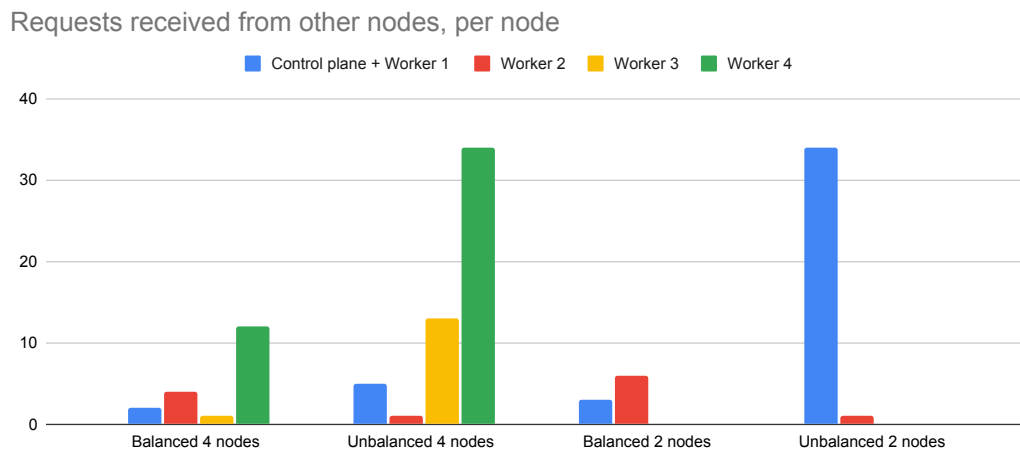


Figure 12: The diagram shows how many requests each node receives from another node per node and setup. In the balanced setups, there is less difference between the received requests per node. In the unbalanced setups, one node always receives far more requests than the other nodes. This creates an imbalance in the setup.

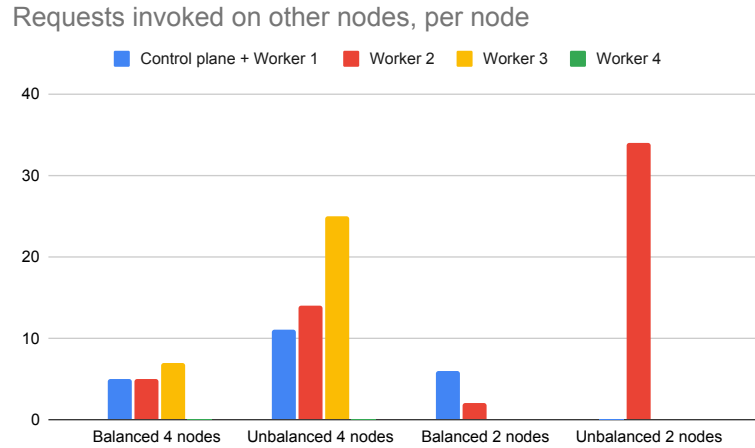


Figure 13: The diagram shows how many times a service invoked another service on another node per node and setup. The balanced setups have a lower amount of invoked requests in total compared to the unbalanced ones. There are also two spikes on worker 3, 4 nodes, and worker 2, on 2 nodes. This creates an imbalance in the setup

4.3 OpenTelemetry

To make the services generate and export traces, auto-instrumentation was used. This works by giving the pods a specific annotation. The OpenTelemetry Kubernetes operator can then inject an init container for the pods with the instrumentation annotation. The init container then injects the necessary code to perform tracing. Each service was instrumented to generate spans and export them to a collector, which was deployed either as a sidecar or daemonset collector. The OpenTelemetry operator was locked on version 0.49, the latest version available when tests were run. The back-end used to store traces was Jaeger, as it is a popular choice for visualizing and storing traces, and it also supports the OpenTelemetry Protocol (OTLP). The OTLP protocol was used to send spans between collectors using gRPC as it is the main protocol built by OpenTelemetry (OpenTelemetry 2024h). When testing on four nodes, Jaeger was placed on the third worker node, and when testing on two nodes, Jaeger was placed on the second worker node. The OpenTelemetry operator, which injects init containers and sidecars, was placed on worker 4 when running with 4 nodes and worker 2 when running with 2 nodes.

4.4 Collector Placement

Two main collector deployment strategies were tested. One used sidecars, and the other used daemonset. Both used a load balancer after the daemonset or sidecar. The load balancer was configured to deterministically route the spans based on their trace ID. After the load balancer, two collectors were placed, and at the end, Jaeger was placed. The load balancer, followed by two collectors, was deployed to keep the scenario as realistic as possible. All collectors deployed were always locked on version 0.85, the latest version, when running experiments. The system architecture and all the tools used to deploy each component in the system can be seen in Figure 14.

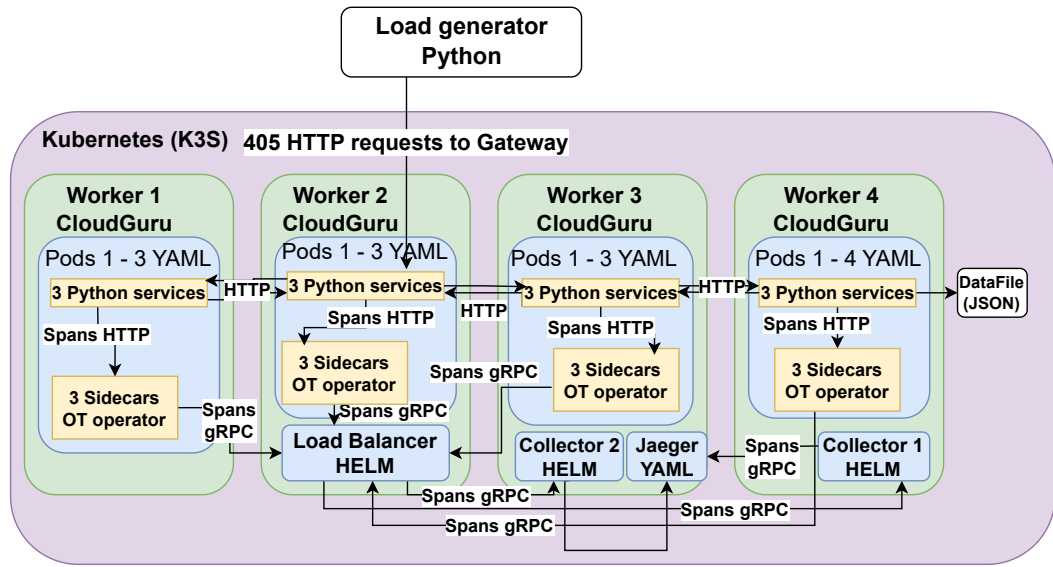


Figure 14: The overall system architecture. The load generator is written in Python, it invokes HTTP requests to the gateway. The worker nodes are hosted on CloudGuru and are orchestrated with the Kubernetes K3s distribution. Each microservice is placed in a pod, they are deployed with YAML files using Kubectl. Each microservice container is written in Python and containerized. The services communicate via HTTP requests. Spans to the sidecar collector are sent using HTTP. The sidecar collectors are deployed by the OpenTelemetry Kubernetes operator. Spans to the load balancer, collectors and Jaeger are sent via gRPC and deployed using HELM charts.

4.4.1 Sidecar Collector

The sidecar collector containers were injected by the OpenTelemetry Operator in each pod annotated with a specific sidecar annotation. The services exported spans to the sidecar in the pod, which then exported to the load balancer. The load balancer exports spans to one of two collectors, which then export to Jaeger. The setup and amount of

spans sent between nodes and pods can be seen in figures 15 and 16. These figures have the pods and services bundled together to help illustrate how it looks like when pods and sidecar collectors are expanded on worker 4, can be seen in Figure 17.

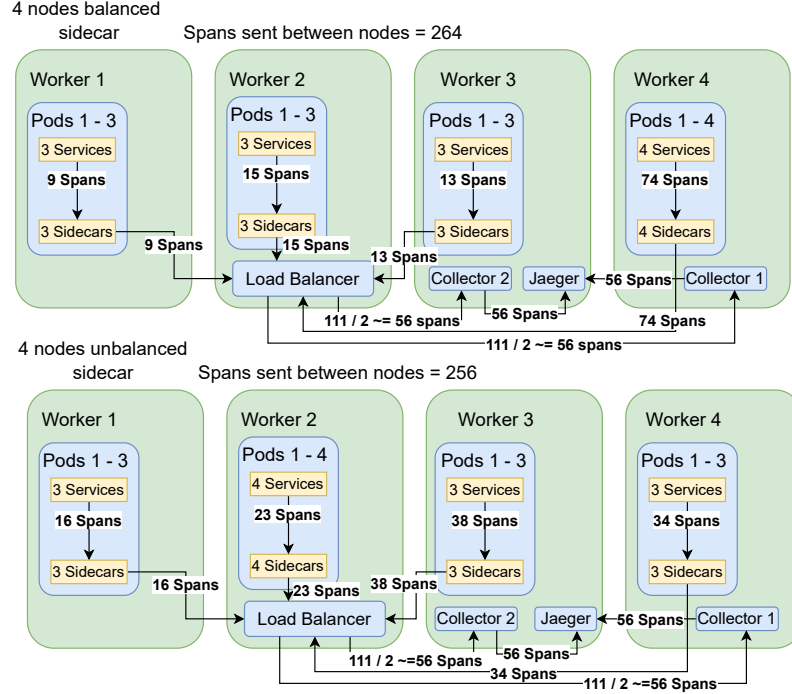


Figure 15: The two setups balanced and unbalanced for the sidecar setup with 4 nodes. The graph shows how many traces each service will generate and where they will be sent. The green boxes are nodes, blue are pods. The services have been bundled up into a single pod to reduce the size of the figure.

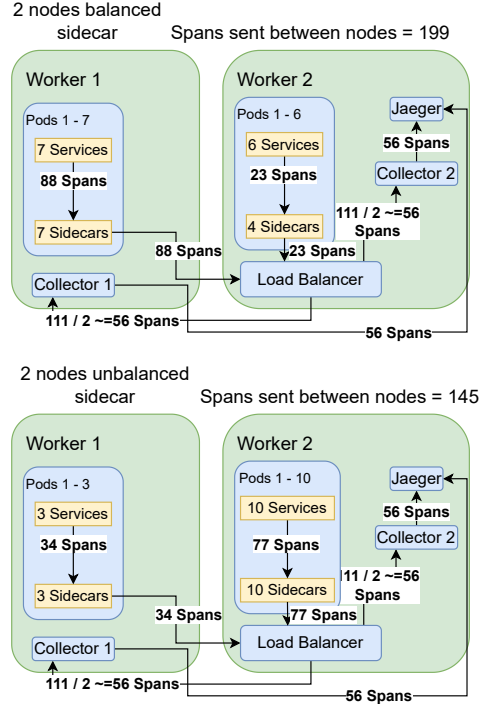


Figure 16: The two setups balanced and unbalanced for the sidecar setup with 2 nodes. The graph shows how many traces each service will generate and where they will be sent. The green boxes are nodes, blue are pods. The services have been bundled up into a single pod to reduce the size of the figure.

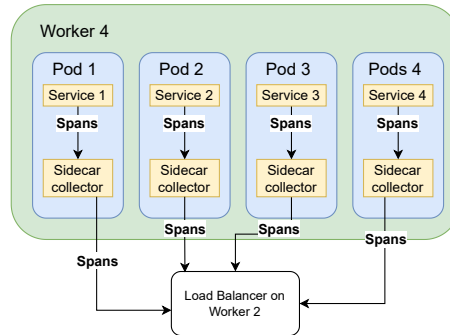


Figure 17: This figure shows how the pods are placed on the balanced setup, 4 nodes on worker 4. Each service is placed in a pod, and each pod has a sidecar collector to which services export spans. The sidecars then export the spans to the load balancer on worker 2.

4.4.2 Daemonset Collector

With the daemonset collector setup, the services would export spans to a daemonset collector deployed on the same node. Like with the sidecar setup, the daemonset collector would then export to a load balancer, which exports to one of two collectors and, lastly, ends up in Jaeger. Where each pod is placed and how traces would travel

between nodes and pods can be seen in figures 18 and 19.

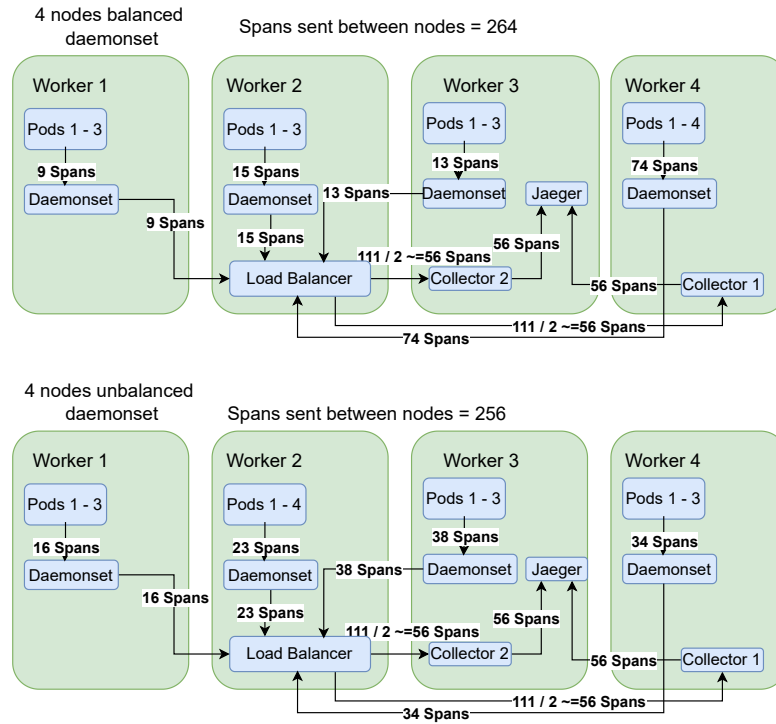


Figure 18: The two setups balanced and unbalanced for the daemonset setup with 4 nodes. The graph shows how many traces each service will generate and where they will be sent. The green boxes are nodes, blue are pods. The services have been bundled up into a single pod to reduce the size of the figure.

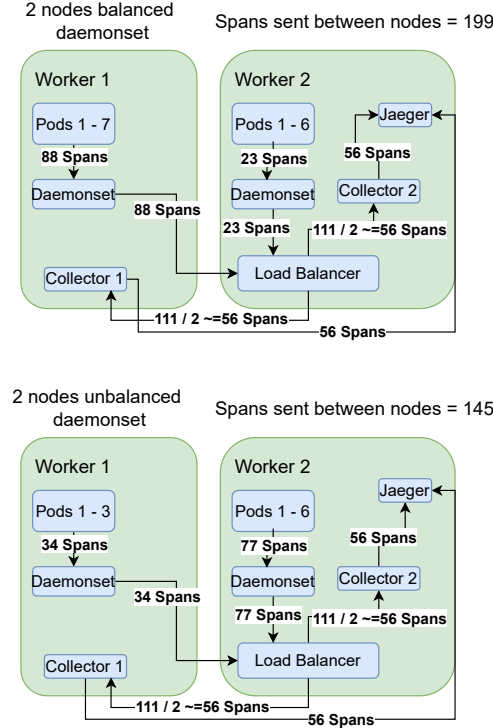


Figure 19: The two setups balanced and unbalanced for the daemonset setup with 4 nodes. The graph shows how many traces each service will generate and where they will be sent. The green boxes are nodes, blue are pods. The services have been bundled up into a single pod to reduce the size of the figure.

4.5 Load Generator

A load generator was implemented to invoke the API gateway in the cluster. The load generator ran three processes in parallel that executed calls to the cluster. After each call, the processes slept for 0.05 seconds. Each process invoked exactly 135 requests to the cluster, creating a total of 405 requests per run. The load generator also logged the latency for each call, as well as the start and end time used for metric querying.

4.6 Test Cases

The different test cases were based on the daemonset setup, sidecar setup, no opentelemetry combined with four nodes, and two nodes combined with unbalanced and balanced. Permuting the setups gives a total of 12 test cases for each node setup (2), each collector setup (3), and each balance (2). Each test case was run 7 times, giving a total of 84 test runs. The metrics collected from each run in the experiment were aggregated to depict a single run, which could later be used as a point in box

plots.

4.7 Metric Collection

Prometheus was used to scrape different metrics in the cluster. Prometheus was placed on the control plane in all of the experiments. A node exporter was also deployed on the other nodes, which exports node metrics. The different metrics scraped and collected were total CPU work seconds, received bytes per node, and the allocated amount of memory. These metrics were all collected per node and aggregated as a single number for each run. The CPU seconds were calculated by measuring the total CPU seconds accumulated at the end of the experiment and subtracting the amount of accumulated CPU seconds at the beginning of the experiment. This gave the number of seconds the CPU had to work for each experiment. Since each node had 2 cores, this number is the sum of each core. The received amount of bytes was calculated by taking the difference between the accumulated amount of received bytes from the end to the start. This gave a number which is how many bytes each node received for one experiment. To guarantee that only the network between nodes was measured, the interface for the NAT network was isolated and measured. The RAM was measured at regular intervals during the experiment and was summed at the end of the experiment, quantifying how much RAM each node had allocated for a run.

Each run and resource measured were placed in a box plot for visualization. The box plots were grouped so that one graph contained the runs for each node grouped by the experiment setup.

5 Results

This chapter presents the results for each setup. The results are presented as box plots. Each figure contains box plots for each setup and worker node, which are presented for each balanced setup. Each worker node is presented as there are resource imbalances among nodes, which are important to present as they describe the behaviour of different system components. The values in the box plots represent one single run, each plot contains 7 runs. The time taken, CPU usage, Network usage, and RAM usage for the setups 2 nodes and 4 nodes, as well as for balanced and unbalanced are presented. These graphs were chosen as they present vital differences between setups and collector deployment methods.

5.1 Time Taken

The load generator measured how long time it would take to invoke all 405 requests to the API-gateway. The no OpenTelemetry setup was always the fastest. The second fastest was daemonset on both 4 and 2 nodes. The sidecar setup was always the slowest. On the 2-node setup, unbalanced was always slower. On the 4-node setup, there was no obvious difference between balanced and unbalanced. The time taken for all experiments can be seen in figures 20 and 21.

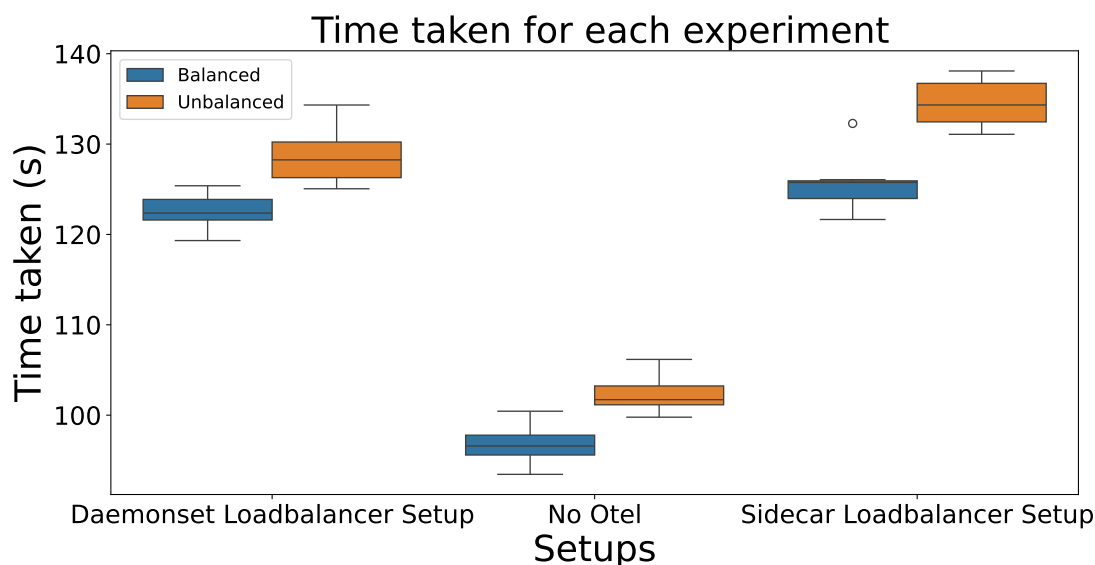


Figure 20: Time taken for each setup to complete the total of 405 invocations for each setup on 2 nodes. For both the balanced and unbalanced scenarios, we observe an average additional execution time of 28.5 % for using the OT framework. In the daemonset setup, unbalanced took 5 % longer than balanced. In the no Otel setup, unbalanced took 8.2 % longer than balanced. In the sidecar setup unbalanced took 10.8 % longer than balanced.

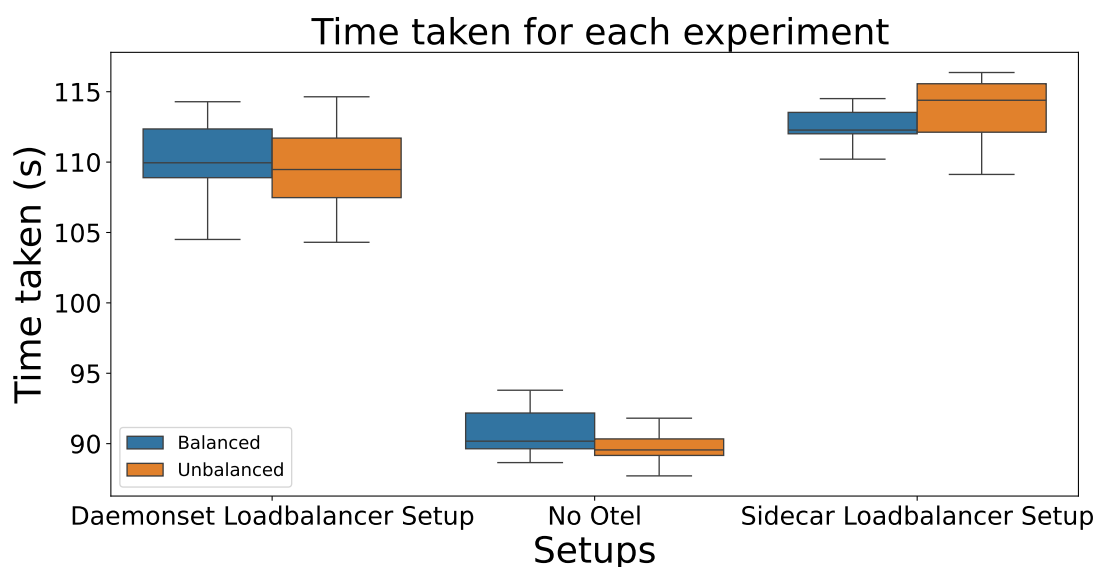


Figure 21: Time taken for each setup to complete the total of 405 invocations for each setup on 4 nodes. For both the balanced and unbalanced scenarios, we observe an average additional execution time of 24.8 % for using the OT framework.

5.2 2 Nodes

This section presents the resources used for each node and resource when running on two nodes.

5.2.1 CPU Usage

The CPU seconds consumed were always the lowest when running without Open-Telemetry. The second lowest was the daemonset setup, and the slowest setup was the sidecar. When running the experiments on balanced, daemonset used in total 56 % more CPU seconds than no otel. Sidecar used 62 % more CPU than no otel, and sidecar used 4 % more than daemonset. In the unbalanced setup, daemonset used 46 % more CPU than no otel. Sidecar used 52 % more CPU than no otel, and sidecar used 4 % more than daemonset. The results for CPU consumption when running on two nodes can be seen in figures 22 and 23.

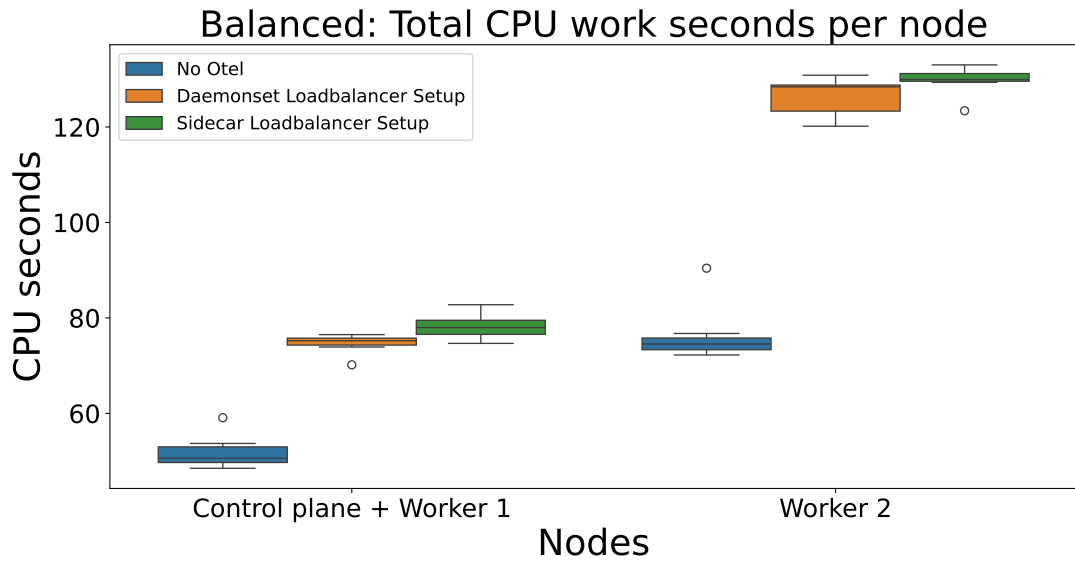


Figure 22: The amount of consumed CPU seconds for each setup and node when running on two nodes with balanced services. On the control plane, daemonset used 43.6 % more than no otel, sidecar used 4.9 % more than daemonset. On worker 2, daemonset used 65 % more than no otel, sidecar used 2.8 % more than daemonset.

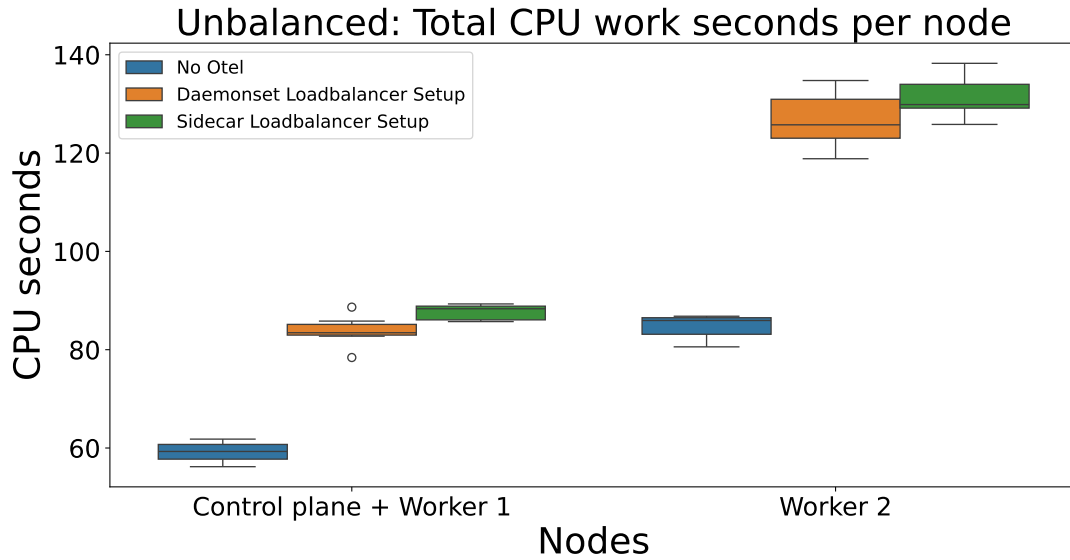


Figure 23: *The amount of consumed CPU seconds for each setup and node when running on two nodes with unbalanced services. On the control plane, daemonset used 41.6 % more than no otel, sidecar used 4.5 % more than daemonset. On worker 2, daemonset used 49.7 % more than no otel, sidecar used 4.5 % more than daemonset.*

5.2.2 Network Usage

No OpenTelemetry always received the lowest amount of bytes on all nodes, both balanced and unbalanced. Sidecar had a higher median than daemonset on both balanced and unbalanced on all nodes except for worker 2 on the unbalanced setup. When running experiments on balanced, daemonset used in total 26 % more than no otel. Sidecar used 28 % more than no otel. In the unbalanced setup, daemonset used 10 % more than no otel. Sidecar used 10 % more than no otel. The results can be seen in figures 24 and 25.

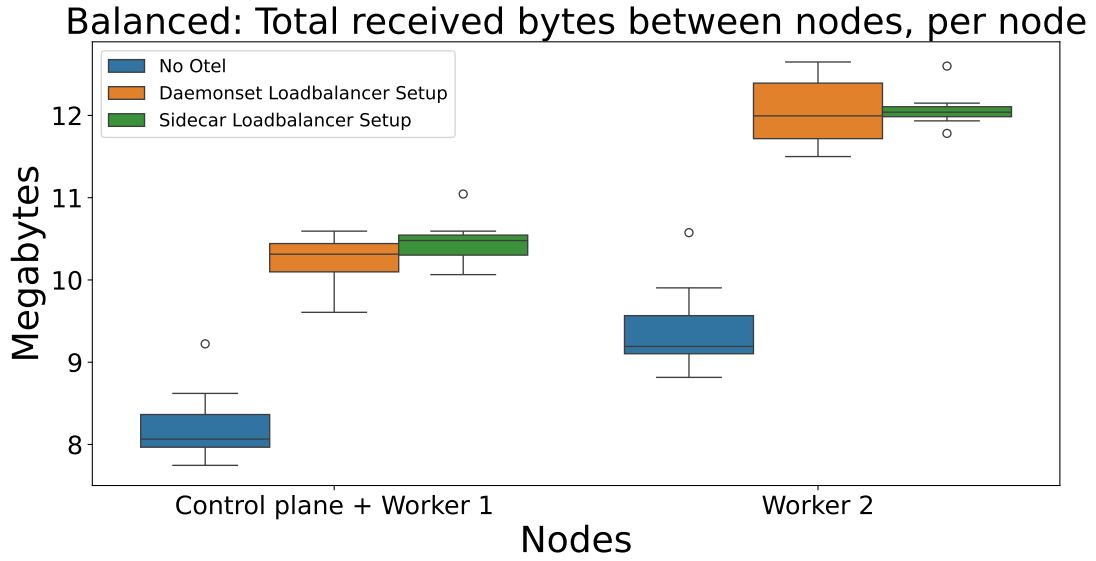


Figure 24: The number of bytes received for each setup and node when running on two nodes with balanced services. On the control plane, daemonset used 24.1 % more than no otel, sidecar used 2.4 % more than daemonset. On worker 2, daemonset used 28 % more than no otel

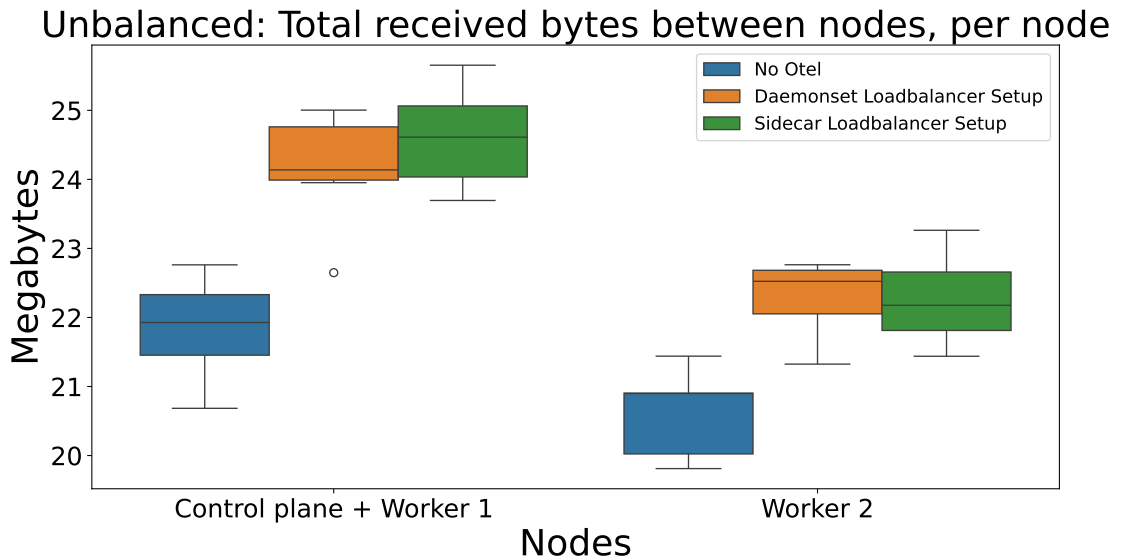


Figure 25: The number of bytes received for each setup and node when running on two nodes with unbalanced services. On the control plane, daemonset used 10.7 % more than no otel, sidecar used 1.7 % more than daemonset. On worker 2, daemonset used 8.4 % more than no otel

5.2.3 Memory Usage

No OpenTelemetry always consumed the lowest amount of memory in all cases on two nodes. Daemonset consumed the second lowest in all cases on two nodes. Sidecar

consumed the highest amount of memory. When running experiments on balanced, daemonset used 57 % more memory than no otel. Sidecar used 80 % more memory than no otel, and sidecar used 14 % more memory than daemonset. In the unbalanced setup, daemonset used 52 % more memory than no otel. Sidecar used 76 % more memory than no otel, and sidecar used 16 % more memory than daemonset. The results can be seen in figures 26 and 27.

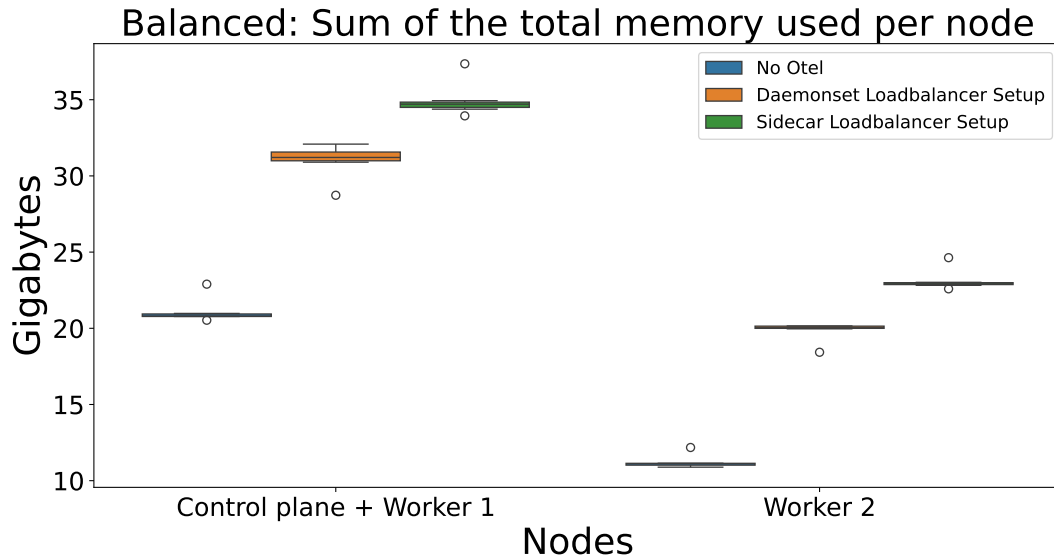


Figure 26: The sum of all allocated bytes when running on two nodes and with balanced services for each node and setup. On the control plane, daemonset used 47.1 % more than no otel, sidecar used 12.7 % more than daemonset. On worker 2, daemonset used 76.9 % more than no otel, sidecar used 16.6 % more than daemonset.

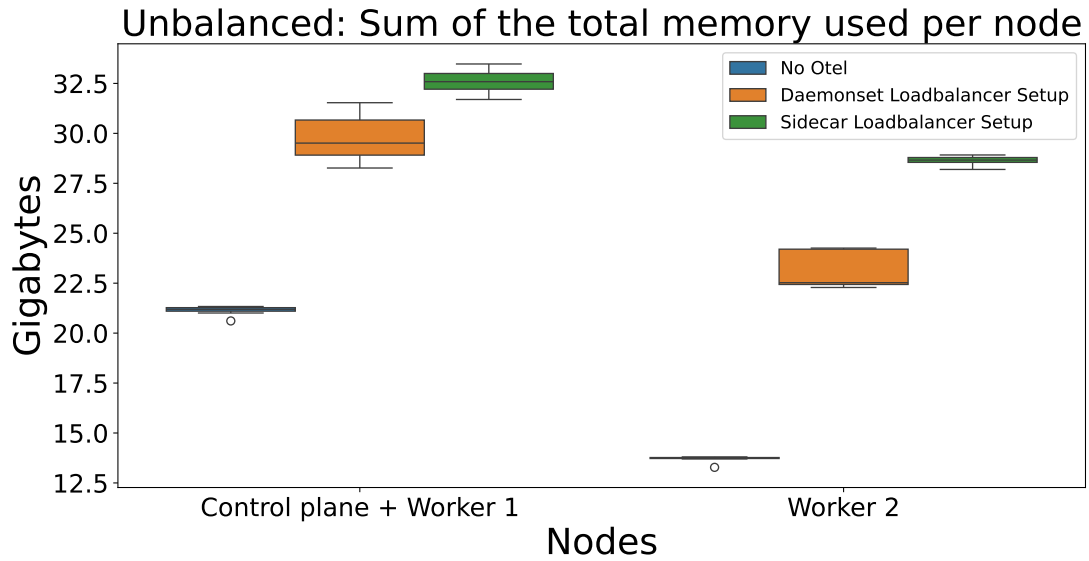


Figure 27: The sum of all allocated bytes when running on two nodes and with unbalanced services for each node and setup. On the control plane, daemonset used 41 % more than no otel, sidecar used 9.5 % more than daemonset.

5.3 4 Nodes

This section presents the resources used for each node and resource when running on four nodes.

5.3.1 CPU Usage

Both on the balanced and unbalanced setup, no OpenTelemetry had the lowest amount of consumed CPU seconds. The second slowest was Daemonset which always had a median lower than the sidecar setup. When running experiments with the balanced setup, daemonset used 42 % more CPU seconds than no otel. Sidecar used 48 % more than no otel, and sidecar used 5 % more than daemonset. In the unbalanced setup, daemonset used 42 % more than no otel. Sidecar used 53 % more than no otel, and sidecar used 8 % more than daemonset. On the unbalanced setup, there is more spread than on the balanced setup. On balanced, worker 4 has more CPU seconds on all setups compared to the other nodes. The results can be seen in figures 22 and 23.

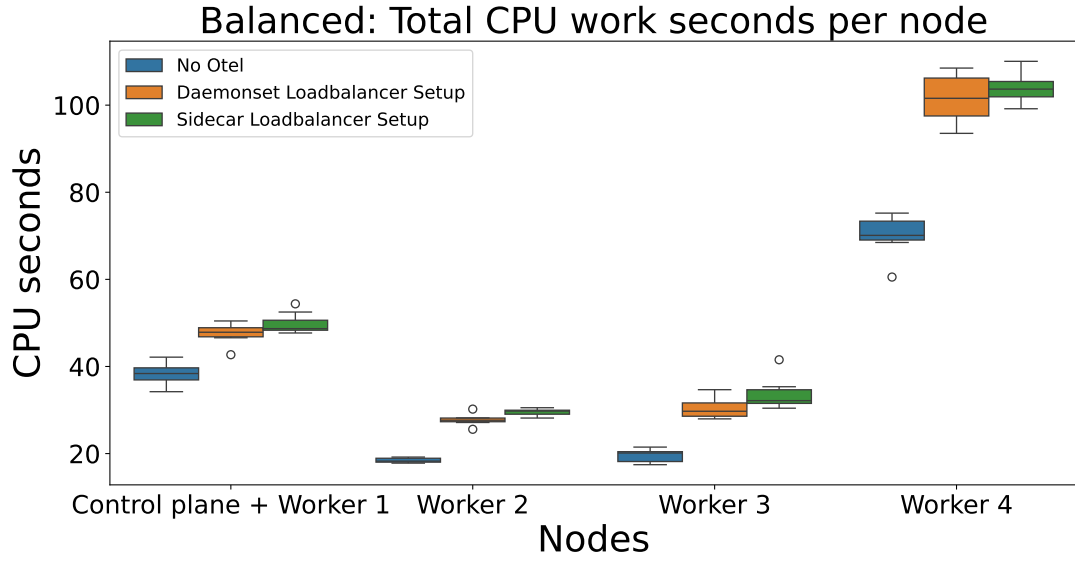


Figure 28: The total amount of consumed CPU seconds per setup and node when running on four nodes with balanced services. On the control plane, daemonset used 24.1 % more than no otel, sidecar used 4.9 % more than daemonset. On worker 2, daemonset used 50.5 % more than no otel, sidecar used 6.2 % more than daemonset. On worker 3, daemonset used 56.1 % more than no otel, sidecar used 11.1 % more than daemonset. On worker 4, daemonset used 44.9 % more than no otel, sidecar used 2.3 % more than daemonset. Worker 4 had higher CPU seconds compared to all other nodes on all deployment methods.

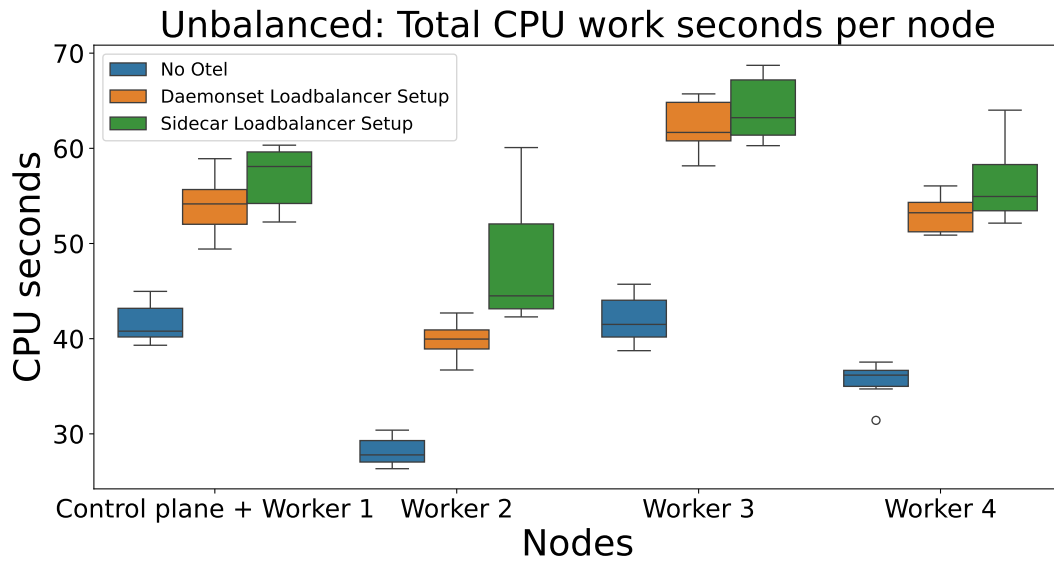


Figure 29: The total amount of consumed CPU seconds per setup and node when running on four nodes with unbalanced services. On the control plane, daemonset used 29.5 % more than no otel, sidecar used 5.4 % more than daemonset. On worker 2, daemonset used 41.5 % more than no otel, sidecar used 20.9 % more than daemonset. On worker 3, daemonset used 48.4 % more than no otel, sidecar used 2.9 % more than daemonset. On worker 4, daemonset used 49.4 % more than no otel, sidecar used 6.3 % more than daemonset.

5.3.2 Network Usage

When running on four nodes, No OpenTelemetry always received the fewest bytes on both balanced and unbalanced nodes. Sidecar received more on the median than the daemonset setup in all cases except for the unbalanced case on worker 3. When running the balanced setup, daemonset received in total 21 % more bytes than no otel. Sidecar received 24 % more than no otel, and sidecar received 3 % more than daemonset. In the unbalanced setup, daemonset received 16 % more than no otel. Sidecar received 17 % more than daemonset. The results can be seen in figures 24 and 25.

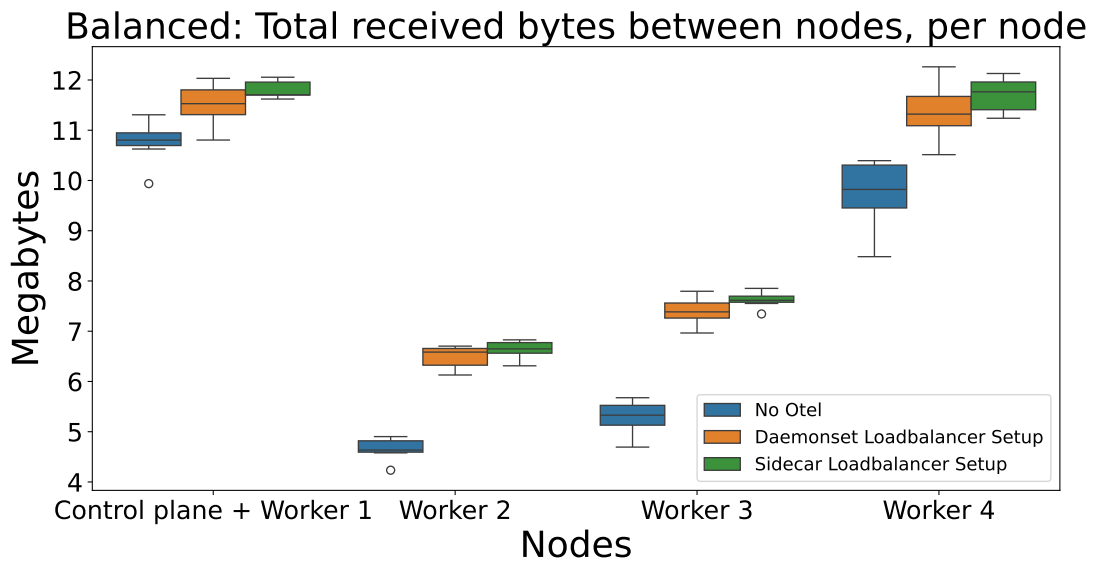


Figure 30: The number of bytes received for each setup and node when running on four nodes with balanced services. On the control plane, daemonset used 7 % more than no otel, sidecar used 2.6 % more than daemonset. On worker 2, daemonset used 39.2 % more than no otel, sidecar used 2.4 % more than daemonset. On worker 3, daemonset used 39.9 % more than no otel, sidecar used 3 % more than daemonset. On worker 4, daemonset used 16.7 % more than no otel, sidecar used 2.8 % more than daemonset.

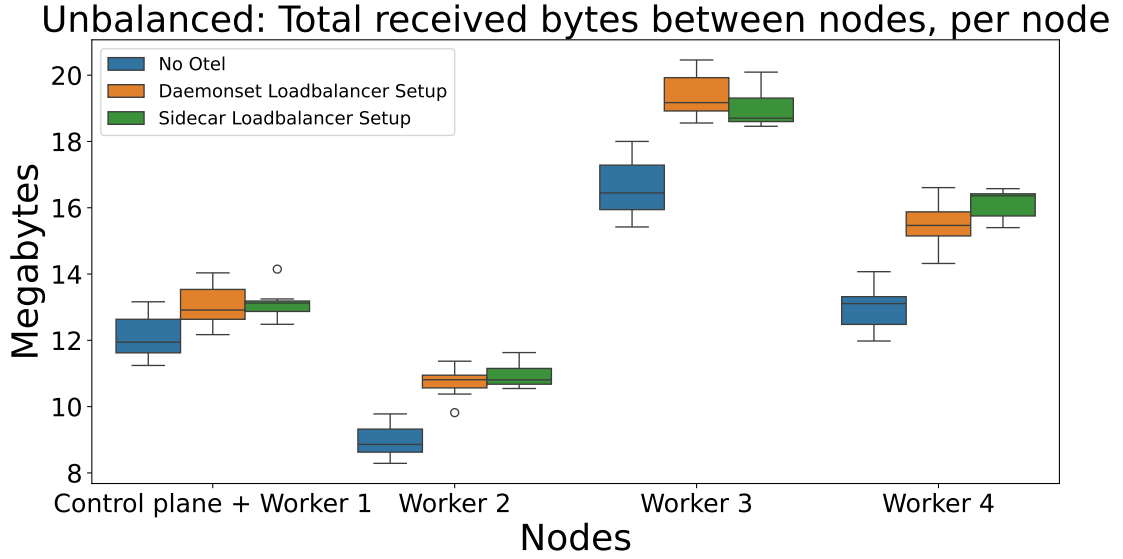


Figure 31: The number of bytes received for each setup and node when running on four nodes with balanced services. On the control plane, daemonset used 7.8 % more than no otel, sidecar used 0.4 % more than daemonset. On worker 2, daemonset used 19.4 % more than no otel, sidecar used 2.2 % more than daemonset. On worker 3, daemonset used 16.8 % more than no otel, daemonset used 2.1 % more than sidecar. On worker 4, daemonset used 19.5 % more than no otel, sidecar used 3.9 % more than daemonset.

5.3.3 Memory Usage

Memory usage was always the lowest for no OpenTelemetry in all cases when running on four nodes, both balanced and unbalanced. Sidecar always used more memory than the daemonset setup in all cases on four nodes. In the balanced setup, daemonset used a total of 38 % more memory than no otel. Sidecar used 50 % more memory than no otel, and sidecar used 9 % more than daemonset. In the unbalanced setup, daemonset used 43 % more memory than no otel. Sidecar used 59 % more than no otel, and sidecar used 10 % more than daemonset. The results can be seen in figures 32 and 33.

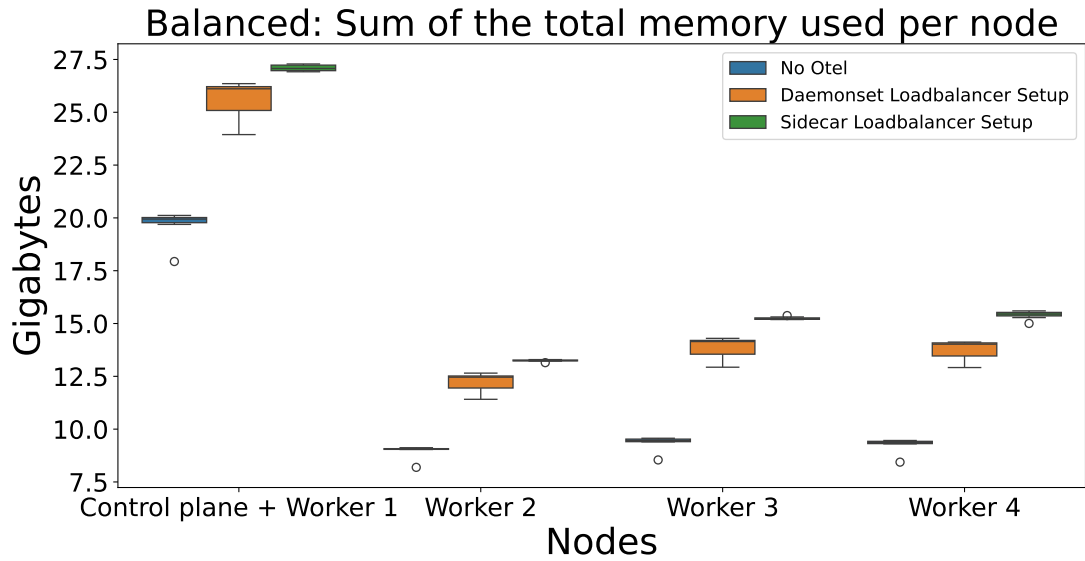


Figure 32: The sum of all allocated bytes when running on four nodes and with balanced services for each node and setup. On the control plane, daemonset used 30.1 % more than no otel, sidecar used 6 % more than daemonset. On worker 2, daemonset used 36.5 % more than no otel, sidecar used 8.5 % more than daemonset. On worker 3, daemonset used 48 % more than no otel, sidecar used 10.2 % more than daemonset. On worker 4, daemonset used 48.5 % more than no otel, sidecar used 12.1 % more than daemonset.

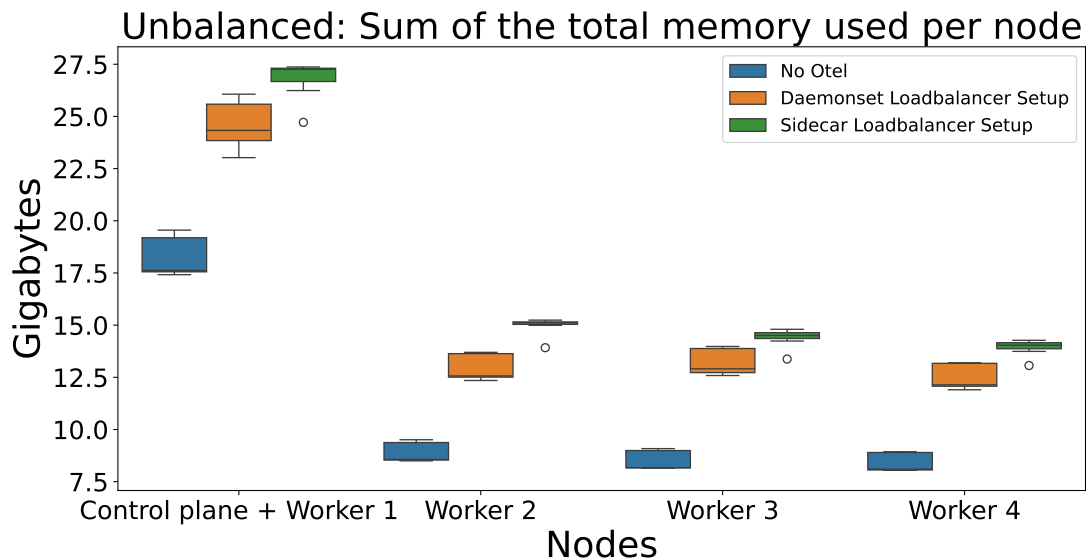


Figure 33: The sum of all allocated bytes when running on four nodes and with balanced services for each node and setup. On the control plane, daemonset used 34.5 % more than no otel, sidecar used 8.7 % more than daemonset. On worker 2, daemonset used 45.7 % more than no otel, sidecar used 15.1 % more than daemonset. On worker 3, daemonset used 55.2 % more than no otel, sidecar used 8.6 % more than daemonset. On worker 4, daemonset used 48.7 % more than no otel, sidecar used 11.1 % more than daemonset.

6 Discussion

This chapter analyzes the results with respect to the research question. It discusses and explains the system's general behavior and the reason why OpenTelemetry generates overhead in certain resources. An explanation is given for why sidecar performed worse in most cases compared to the daemonset setup. Also, a comparison of the differences between 2 and 4 nodes, balanced and unbalanced, is presented.

6.1 OpenTelemetry Overhead

Having OpenTelemetry generate and collect traces did add overhead in all cases. Using the daemonset setup increased the CPU seconds by 46.5 % on average. The received bytes also increased by 18.25 % on average. Each time an instrumented service either invokes or gets invoked by another service, it must generate a span and export it. Both the generation and exporting consume some CPU seconds. Then the span has to be received and processed at three collectors, and finally stored in Jaeger. This mechanism adds to both the CPU and received bytes when using OpenTelemetry. The increased CPU time for using OpenTelemetry did, in turn, increase the total time taken for the request using OpenTelemetry. The total memory used also increased when using OpenTelemetry. When comparing the daemonset setup to no OpenTelemetry, it increased by 47.5 % on average. This increases as each service's process increases its memory when instrumented, and each collector process consumes memory. The spans being generated also accumulate some memory for each run.

6.2 Sidecar and Daemonset

In most cases, sidecar performed worse for every resource than daemonset. On average, the sidecar consumed 5.25 % more CPU seconds and received 1.5 % more bytes compared to the daemonset. In the sidecar setup, there is one sidecar collector process for each service in the node. Since each sidecar opens up one connection each to the loadbalancer, this creates multiple parallel connections to the loadbalancer in total, which always is on another node, except for the node that contains the loadbalancer. There are 13 services, creating 13 parallel connections to a single loadbalancer. In the daemonset setup, there is one collector per node, which services export spans to. This creates a total of 2 or 4 connections to the loadbalancer, depending on the

setup. However, it creates multiple parallel connections to each daemonset collector, but this number is smaller and guaranteed to be on the same node. The daemonset collectors then open up their connection to the loadbalancer and send all the spans from the services on the node through the same connection. Having too many processes export to the loadbalancer can cause both the receiving and exporting ends to start slowing down due to CPU swapping and thrashing. It also causes more data to be sent over the network. As there are more connections in total with the sidecar setup, the overhead of the collector connections get amplified. Having more powerful CPUs and distributing the collectors differently might reduce this effect. The sidecar setup consumed 12.25 % more memory compared to the daemonset setup. This occurs since the sidecar setup needs 13 sidecar processes running, while the daemonset setup only needs 2 or 4, depending on the setup.

In Figure 25, on worker 2, daemonset receives fewer bytes than sidecar. This is because there is more than half the amount of spans sent to worker 2 on unbalanced than balanced. There are also 3 more services on worker 1 on unbalanced, which is seen in Figure 19. This reduction in spans sent to worker 2 on and the lower amount of sidecars lowers the effect of the sidecar overhead.

Figure 31 shows that on 4 nodes, unbalanced, worker 3, daemonset receives more than sidecar. This might be because this worker holds the service GE-G, which generates 33 spans. The two other services on the node generate two spans each. This makes the sidecar setup very much like a daemonset, as almost all spans come from one service. This reduces the difference between the overhead of the two setups.

Even though sidecar may perform worse compared to daemonset in most cases and resources, it is worth noting that it can be more flexible when configuring different pipelines for different telemetry, as discussed in Section 3.8. It is probably more likely to use a combination of both in a real-world example. One service might, for example, generate a large amount of spans, or it might not be interesting to implement tracing on that service. Services with different needs and requirements could, in that case, be configured to use their own sidecar collectors, whereas services that are more or less the same could export to the same daemonset collector. It is also worth noting that these results are for tracing only, OpenTelemetry can also be used to collect metrics and logs from services and nodes, as discussed in Section 3.8. These are telemetry that might need more specific configurations depending on which services are being instrumented.

6.3 2 and 4 Nodes

Having more nodes did reduce the total time taken for every setup, both balanced and unbalanced. This is because the majority of the work done per request is related to the services requesting and responding to the HTTP requests. Distributing this work and doubling the number of nodes and CPU capacity reduced the total time taken. Even though 4 nodes send about 33 % more spans than 2 nodes, between nodes on balanced and 77 % more spans on unbalanced, it makes out a lesser proportion of the work. With the given setup, where all the spans need to reach a final back-end, scaling up the nodes will always increase the number of spans that need to be sent between nodes.

As presented in Section 5.2.1, both daemonset and sidecar increased the CPU usage by about 50 %. We expect this percentage to be lower with more CPU-intensive and longer-running tasks. In this study's case, the services' baseline work was comparatively low compared to the OpenTelemetry overhead, creating a worst-case for the resource overhead. As the proportion of the work that is not overhead grows, the percentage of the work that is overhead will reduce. However, in this case, it was preferred, as it isolated the overhead.

6.4 Balanced and Unbalanced

In the unbalanced setup, the amount of received bytes between nodes is always higher than in the balanced setup. In most cases, for both 2 and 4 nodes, the amount is at least doubled. This is expected since the design of the unbalanced was to maximize the amount of requests sent between nodes.

On 4 nodes, the unbalanced setup was, in some cases, faster than the balanced. This is because, on balanced 4 nodes, worker 4 had to process 77 % of all the requests. This created a bottleneck node in the system, slowing it down. In Figure 31, on worker 3, no Otel receives more than worker 4, which might seem wrong as worker 4 receives more requests from other nodes than worker 3. However, if we look at both figures 13 and 12, the sum of requests invoked and received is larger on worker 3 than on worker 4. Since an HTTP request also includes a response message, the received bytes can become larger on worker 3 even though it has fewer requests received. This is also the reason why worker 3 has more CPU seconds than worker 4 in Figure 29. We also see in Figure 30 that worker 1 has more received bytes than worker 4, which might seem wrong as it has fewer requests and invocations than worker 4. This is because

Prometheus is deployed on worker 1. Prometheus receives metrics from the nodes at regular intervals, increasing the received bytes on this node.

When comparing the CPU seconds between balanced and unbalanced, it can be noted that there was a larger spread on the unbalanced setup, especially on 4 nodes. In Figure 29, we can see that there is a larger spread compared to the values in the balanced setup. On worker 2, there is a large spread on the sidecar setup. This might be because the worker holds the load balancer, which all spans must pass through. The load balancer will have the highest load on the unbalanced sidecar setup, as it is when it receives the most spans from the most amount of collectors.

7 Conclusion

This study concluded that there is a difference in resource consumption when it comes to different collector deployment patterns in a cloud environment. Resource consumption also changed for both sidecar and daemonset when using more or fewer nodes and running experiments with balanced or unbalanced service placement on nodes. The sidecar setup increased CPU, network, and RAM consumption in most cases compared to the daemonset setup. RAM usage was the biggest increase in resource usage when using sidecar compared to daemonset. More memory is required since the sidecar uses one collector instance per service instead of one per node. The CPU increase can be explained by having more processes compared to daemonset. It also requires more parallel connections to the loadbalancer, which creates network overhead, increasing the total bytes received. It was also concluded that both sidecar and daemonset setup increased resource consumption in all cases compared to running the experiments with no OpenTelemetry implemented. However, it should be noted that a combination of both methods is likely needed in a real-world scenario. Configuring the telemetry pipeline using sidecars is also easier since each service gets its own collector.

7.1 Future Work

As of now, it is up to the implementer of the OpenTelemetry framework to determine where to put collectors. This might be inefficient and time-consuming. It could be possible to automate this process based on the Kubernetes cluster attributes. It could even be made dynamically so that the deployment changes when the system load is moved around in the cluster. Future work on this version could be done by testing in another cloud environment with more resources. The same experiment could be run using K8s to see if there is any difference compared to K3s. This would allow for a fairer comparison of different methods. Comparisons could also be made on different configurations in OpenTelemetry. For example, this project mainly utilized gRPC as a receiver and exporter on the collectors, so one could compare the efficiency of gRPC over HTTP. There are also different propagation protocols, that is, the protocol used to send span contexts between services. This study only used OTLP, but many other protocols are built into OpenTelemetry. Different batching settings could also be used. The collectors batch processor holds the spans for some time or until enough spans have been received before it exports them. The time and number

of spans can easily be lowered or higher, these changes might favor some methods. This study only looked at the tracing part of OpenTelemetry, it would be interesting to compare the methods daemonset or sidecar when collecting other telemetry, such as logs and metrics as well. A comparison could also be made between manual and automatic instrumentation, this study only used automatic instrumentation. It could also be tested for different programming languages, as the automatic instrumentation mechanisms vary between languages.

7.2 Recommendations

This section addresses problems that occurred and how they can be avoided in future work. The automatic instrumentation in OpenTelemetry does not work for all package versions in the language it is being implemented. For example, when instrumenting a Flask server in Python, the Flask version has to be less than 3. Otherwise, no traces will be generated, and no error will be displayed. The versions that are supported can be found in the OpenTelemetry documentation. There were also regular updates in the OpenTelemetry project, which affected the performance and functionality of the system, it is therefore necessary to lock the OpenTelemetry collectors and operator in some version.

There are two main ways of deploying the collectors and operators in a Kubernetes cluster. The first method utilizes regular YAML files, which are applied with Kubectl. However, a more configurable and easy way to deploy it is to use the Helm charts created by OpenTelemetry. They allow for easier configurations and version management.

References

- Aguilera, Marcos K et al. (2003). “Performance debugging for distributed systems of black boxes”. In: *ACM SIGOPS Operating Systems Review* 37.5, pp. 74–89.
- Amazon web services (n.d.). *What is a RESTful API?* <https://aws.amazon.com/what-is/restful-api/>. Accessed:2024-02-29.
- Bento, Andre et al. (2021). “Automated analysis of distributed tracing: Challenges and research directions”. In: *Journal of Grid Computing* 19, pp. 1–15.
- Chakraborty et al. (2021). *Part I. Architecture of Modern Monitoring Systems for Cloud Native Applications*. URL: <https://app.knovel.com/hotlink/khtml/id:kt0134CV09/monitoring-cloud-native/part-i-architecture-modern>.
- Cinque, Marcello, Raffaele Della Corte, and Antonio Pecchia (2022). “Microservices Monitoring with Event Logs and Black Box Execution Tracing”. In: *IEEE Transactions on Services Computing* 15.1, pp. 294–307. DOI: 10.1109/TSC.2019.2940009.
- Dragoni, Nicola et al. (2017). “Microservices: Yesterday, Today, and Tomorrow”. In: *Present and Ulterior Software Engineering*. Ed. by Manuel Mazzara and Bertrand Meyer. Cham: Springer International Publishing, pp. 195–216. ISBN: 978-3-319-67425-4. DOI: 10.1007/978-3-319-67425-4_12. URL: https://doi.org/10.1007/978-3-319-67425-4_12.
- Joy, Ann Mary (2015). “Performance comparison between Linux containers and virtual machines”. In: *2015 International Conference on Advances in Computer Engineering and Applications*, pp. 342–346. DOI: 10.1109/ICACEA.2015.7164727.
- Nasdaq (2021). *What is a Clearinghouse? A Simple Explanation for Investors*. [Online; accessed 2-April-2024]. URL: <https://www.nasdaq.com/articles/what-is-a-clearinghouse-a-simple-explanation-for-investors-2021-02-05>.
- OpenTelemetry (2023a). *Manual*. [Online; accessed 14-February-2024]. URL: <https://opentelemetry.io/docs/concepts/instrumentation/manual/>.
- (2023b). *Sampling*. [Online; accessed 15-February-2024]. URL: <https://opentelemetry.io/docs/concepts/sampling/>.
- (2024a). *Agent*. [Online; accessed 16-February-2024]. URL: <https://opentelemetry.io/docs/collector/deployment/agent/>.

- OpenTelemetry (2024b). *Automatic*. [Online; accessed 14-February-2024]. URL: <https://opentelemetry.io/docs/concepts/instrumentation/automatic/>.
- (2024c). *Collector*. [Online; accessed 27-February-2024]. URL: <https://opentelemetry.io/docs/collector/deployment/gateway/>.
 - (2024d). *Configuration*. [Online; accessed 15-February-2024]. URL: <https://opentelemetry.io/docs/collector/configuration/#receivers>.
 - (2024e). *Gateway*. [Online; accessed 16-February-2024]. URL: <https://opentelemetry.io/docs/collector/deployment/gateway/>.
 - (2024f). *No Collector*. [Online; accessed 15-February-2024]. URL: <https://opentelemetry.io/docs/collector/deployment/no-collector/>.
 - (2024g). *OpenTelemetry Operator for Kubernetes*. [Online; accessed 29-February-2024]. URL: <https://opentelemetry.io/docs/kubernetes/operator/>.
 - (2024h). *OTLP Specification 1.3.1*. [Online; accessed 09-May-2024]. URL: <https://opentelemetry.io/docs/specs/otlp/>.
 - (2024i). *What is OpenTelemetry?* [Online; accessed 14-February-2024]. URL: <https://opentelemetry.io/docs/what-is-opentelemetry/>.
- Perdanaputra, Abram and Achmad Imam Kistijantoro (2020). “Transparent Tracing System on gRPC based Microservice Applications Running on Kubernetes”. In: *2020 7th International Conference on Advance Informatics: Concepts, Theory and Applications (ICAICTA)*, pp. 1–5. DOI: 10.1109/ICAICTA49861.2020.9429054.
- Potdar, Amit M et al. (2020). “Performance Evaluation of Docker Container and Virtual Machine”. In: *Procedia Computer Science* 171. Third International Conference on Computing and Network Communications (CoCoNet’19), pp. 1419–1428. ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2020.04.152>. URL: <https://www.sciencedirect.com/science/article/pii/S1877050920311315>.
- Rabkin, Ariel and Randy Katz (2010). “Chukwa: a system for reliable {Large-Scale} log collection”. In: *24th Large Installation System Administration conference (LISA 10)*.
- Reichelt, David Georg, Stefan Kühne, and Wilhelm Hasselbring (2021). “Overhead Comparison of OpenTelemetry, inspectIT and Kieker”. In.
- Rensin, David (2015). *Kubernetes*. O’Reilly Media, Incorporated.

- Slingerland, Cody (n.d.). *K3s Vs K8s: What's The Difference? (And When To Use Each)*.
<https://www.cloudzero.com/blog/k3s-vs-k8s/>. Accessed:2024-02-13.
- Turnbull, James (2014). *The Docker Book: Containerization is the new virtualization*. James Turnbull.
- Van Steen, Maarten and Andrew S Tanenbaum (2016). "A brief introduction to distributed systems". In: *Computing* 98, pp. 967–1009.
- W3C (2021). *Trace Context*. [Online; accessed 15-February-2024]. URL: <https://www.w3.org/TR/trace-context/>.
- Walker, James (2024). *13 Most Useful Container Orchestration Tools in 2024*. [Online; accessed 2-April-2024]. URL: <https://spacelift.io/blog/container-orchestration-tools>.
- Wikimedia Commons (2022). *File:Docker-containerized-and-vm-transparent-bg.png* — *Wikimedia Commons, the free media repository*. [Online; accessed 13-February-2024]. URL: <https://commons.wikimedia.org/w/index.php?title=File:Docker-containerized-and-vm-transparent-bg.png&oldid=673860820>.



UMEÅ UNIVERSITY